claude-windows / 56b7f849-5d56-4770-93e0-b3b4da943700

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\56b7f849-5d56-4770-93e0-b3b4da943700\subagents\workflows\wf_d88615a4-f56\agent-a46cae3f0f615dfc3.jsonl`
- SHA-256: `9f5a1c8c7ebb6661aa203138842f840915b549e638dd4e49c2d4c8e313cd9988`
- Source modified: `2026-06-07T09:12:16+00:00`
- Imported at: `2026-07-05T16:48:27+00:00`
- project: `wf_d88615a4-f56`
- session_id: `56b7f849-5d56-4770-93e0-b3b4da943700`

Transcript

1. user (2026-06-07T09:10:30.787Z)

Read these files in C:/Users/avidu/Projects/badminton-highlight-indexer and produce a DENSE, anchor-rich CODE_MAP chunk for the "Eval & calibration" subsystem.
Use the format: bullet per file/symbol with anchors written as @path::symbol where @path is the repo-relative path EXACTLY as listed (the current layered layout). Capture: key classes/functions/consts; data contracts (mark with §); registries / plugin extension points (mark with ?); and footguns/gotchas (mark with ?). Be precise ? only list symbols that actually exist in the files. If a path is a directory (e.g. tests/), enumerate its files and map each test to the subsystem it covers.
Files/paths: backend/eval/metrics.py, backend/eval/annotations.py, backend/eval/calibration.py, backend/eval/calibrate_wasb.py, backend/eval/calibrate_local.py, backend/eval/export_wasb_segments.py, backend/eval/batch.py, backend/eval/harness.py, backend/eval/training.py, backend/eval/classifier.py, backend/eval/regression.py, backend/eval/distill_local.py, backend/eval/gemini_refine.py, backend/eval/audio/e1_probe.py

2. assistant (2026-06-07T09:10:33.446Z)

I'll read all the files in the "Eval & calibration" subsystem to produce a dense, anchor-rich CODE_MAP chunk.

3. user (2026-06-07T09:10:34.477Z)

```
1       """Pure temporal interval-matching metrics for rally evaluation.
2
3       No I/O, no DB, no global state ? every function here is deterministic and unit
4       tested. An "interval" is a `(start, end)` tuple of floats in seconds with
5       start < end. We score a list of *predicted* rally intervals against a list of
6       *ground-truth* intervals via temporal IoU and one-to-one greedy matching.
7       """
8
9       from dataclasses import dataclass, field
10      from typing import List, Tuple, Optional
11
12      Interval = Tuple[float, float]
13
14
15      def interval_iou(a: Interval, b: Interval) -> float:
16          """Temporal intersection-over-union of two intervals.
17
18          Returns 0.0 when they don't overlap (or either is degenerate). IoU is
19          overlap / union, both measured as durations on the timeline.
20          """
21          a_start, a_end = a
22          b_start, b_end = b
23          if a_end <= a_start or b_end <= b_start:
```

```
            return 0.0
        inter = max(0.0, min(a_end, b_end) - max(a_start, b_start))
        if inter <= 0.0:
            return 0.0
        union = (a_end - a_start) + (b_end - b_start) - inter
        return inter / union if union > 0 else 0.0


    @dataclass
    class Match:
        """One matched predicted?ground-truth pair."""
        pred_index: int
        gt_index: int
        iou: float


    @dataclass
    class MatchResult:
        """Outcome of matching predictions to ground truth at a given IoU threshold."""
        matches: List[Match] = field(default_factory=list)
        unmatched_pred_indices: List[int] = field(default_factory=list)  # false positives
        unmatched_gt_indices: List[int] = field(default_factory=list)    # false negatives
(misses)


    def match_intervals(preds: List[Interval], gts: List[Interval],
                        iou_threshold: float = 0.5) -> MatchResult:
        """Greedy one-to-one matching of predictions to ground truth by descending IoU.

        Every candidate (pred, gt) pair with IoU >= threshold is considered; pairs
        are claimed highest-IoU first, and each prediction/GT can be used once. This
        keeps a single prediction from being credited for two ground-truth rallies
        (and vice versa).
        """
        candidates = []
        for pi, p in enumerate(preds):
            for gi, g in enumerate(gts):
                iou = interval_iou(p, g)
                if iou >= iou_threshold:
                    candidates.append((iou, pi, gi))
        # Highest IoU first; ties broken by indices for determinism.
        candidates.sort(key=lambda c: (-c[0], c[1], c[2]))

        used_pred, used_gt = set(), set()
        matches: List[Match] = []
        for iou, pi, gi in candidates:
            if pi in used_pred or gi in used_gt:
                continue
            used_pred.add(pi)
            used_gt.add(gi)
            matches.append(Match(pred_index=pi, gt_index=gi, iou=iou))

        unmatched_pred = [i for i in range(len(preds)) if i not in used_pred]
        unmatched_gt = [i for i in range(len(gts)) if i not in used_gt]
        return MatchResult(matches=matches,
                           unmatched_pred_indices=unmatched_pred,
                           unmatched_gt_indices=unmatched_gt)


    @dataclass
    class Score:
        """Aggregate metrics for one (predictions vs ground-truth) evaluation."""
        iou_threshold: float
        num_pred: int
        num_gt: int
```

```
88          true_positives: int
89          false_positives: int
90          false_negatives: int
91          precision: float
92          recall: float
93          f1: float
94          mean_iou: float            # over matched pairs (0.0 if none)
95          mean_start_error: float    # mean |pred_start - gt_start| over matches, seconds
96          mean_end_error: float      # mean |pred_end - gt_end| over matches, seconds
97
98          def as_dict(self) -> dict:
99              return self.__dict__.copy()
100
101
102     def score(preds: List[Interval], gts: List[Interval],
103               iou_threshold: float = 0.5,
104               match_result: Optional[MatchResult] = None) -> Score:
105         """Compute precision/recall/F1, mean matched IoU, and boundary errors.
106
107         Pass a precomputed `match_result` to avoid re-matching (e.g. when also
108         extracting the failure lists); otherwise matching is done here.
109         """
110         mr = match_result if match_result is not None else match_intervals(preds, gts,
iou_threshold)
111
112         tp = len(mr.matches)
113         fp = len(mr.unmatched_pred_indices)
114         fn = len(mr.unmatched_gt_indices)
115
116         precision = tp / (tp + fp) if (tp + fp) > 0 else 0.0
117         recall = tp / (tp + fn) if (tp + fn) > 0 else 0.0
118         f1 = (2 * precision * recall / (precision + recall)) if (precision + recall) > 0
else 0.0
119
120         if mr.matches:
121             mean_iou = sum(m.iou for m in mr.matches) / tp
122             mean_start_err = sum(abs(preds[m.pred_index][0] - gts[m.gt_index][0]) for m in
mr.matches) / tp
123             mean_end_err = sum(abs(preds[m.pred_index][1] - gts[m.gt_index][1]) for m in
mr.matches) / tp
124         else:
125             mean_iou = mean_start_err = mean_end_err = 0.0
126
127         return Score(
128             iou_threshold=iou_threshold,
129             num_pred=len(preds), num_gt=len(gts),
130             true_positives=tp, false_positives=fp, false_negatives=fn,
131             precision=precision, recall=recall, f1=f1,
132             mean_iou=mean_iou, mean_start_error=mean_start_err, mean_end_error=mean_end_err,
133         )
134
135
136     def pr_curve(preds: List[Interval], gts: List[Interval],
137                  thresholds: Optional[List[float]] = None) -> List[Score]:
138         """Score across a sweep of IoU thresholds (default 0.1..0.9 step 0.1).
139
140         Lets you see strict-vs-lenient behaviour rather than a single operating point.
141         """
142         if thresholds is None:
143             thresholds = [round(0.1 * i, 1) for i in range(1, 10)]
144         return [score(preds, gts, iou_threshold=t) for t in thresholds]
145
```

4. user (2026-06-07T09:10:34.864Z)

```
    """Load hand-annotated ground-truth rally timestamps from CSV.

    Annotation file format (one file per video):

        start,end
        00:01:12,00:01:39
        102.5,131.0

    - A header row `start,end` is optional (auto-detected and skipped).
    - Each timestamp is either decimal seconds (`102.5`) or colon-separated
      `MM:SS` / `HH:MM:SS` (`00:01:12`). Both forms may be mixed in one file.
    - Blank lines and `#` comment lines are ignored.
    """

    import csv
    import logging
    from typing import List, Tuple

    logger = logging.getLogger(__name__)

    Interval = Tuple[float, float]


    def parse_timestamp(value: str) -> float:
        """Parse a timestamp string to float seconds.

        Accepts decimal seconds or colon-separated MM:SS / HH:MM:SS.
        Raises ValueError on anything unparseable so bad annotations fail loudly.
        """
        value = value.strip()
        if not value:
            raise ValueError("empty timestamp")
        if ":" in value:
            parts = value.split(":")
            if len(parts) > 3:
                raise ValueError(f"too many ':' segments in timestamp '{value}'")
            total = 0.0
            for part in parts:                # most-significant first (H, M, S)
                total = total * 60 + float(part)
            return total
        return float(value)


    def load_annotations(csv_path: str) -> List[Interval]:
        """Load and validate ground-truth rally intervals from a CSV file.

        Returns intervals sorted by start time. Raises ValueError on malformed rows
        (bad timestamp, start >= end) so labeling mistakes surface immediately rather
        than silently skewing scores.
        """
        intervals: List[Interval] = []
        with open(csv_path, newline="") as f:
            reader = csv.reader(f)
            for lineno, row in enumerate(reader, start=1):
                if not row:
                    continue
                first = row[0].strip()
                if not first or first.startswith("#"):
                    continue
                # Skip an optional header row.
                if first.lower() in ("start", "start_time"):
                    continue
                if len(row) < 2:
                    raise ValueError(f"{csv_path}:{lineno}: expected 'start,end', got
{row!r}")
```

```
                try:
                    start = parse_timestamp(row[0])
                    end = parse_timestamp(row[1])
                except ValueError as e:
                    raise ValueError(f"{csv_path}:{lineno}: {e}") from e
                if start >= end:
                    raise ValueError(f"{csv_path}:{lineno}: start ({start}) >= end ({end})")
                intervals.append((start, end))

        intervals.sort(key=lambda iv: iv[0])
        logger.info("Loaded %d ground-truth rallies from %s", len(intervals), csv_path)
        return intervals


    def write_scaffold(csv_path: str) -> None:
        """Write an empty annotation file with just the header, for hand-filling."""
        with open(csv_path, "w", newline="") as f:
            writer = csv.writer(f)
            writer.writerow(["start", "end"])

```

5. user (2026-06-07T09:10:35.810Z)

```
    """Quantify rally-segmentation improvement ? and guard against overfitting.

    This is the measurement layer for per-video / per-corpus calibration. It answers
    two questions honestly:

    1. **"Did precision improve, by how much?"** ? `improvement_report` scores a
       baseline config vs a tuned config on the same ground truth and reports the
       before/after metrics and the % delta.

    2. **"Is that gain real or overfit?"** ? calibration must be measured on data the
       tuning never saw, otherwise you're just memorising. Two protocols:
       - `cross_validate` ? k-fold over one video's rallies: pick the config on the
         train folds, score on the held-out fold. The **generalization_gap**
         (train ? test) is the overfit alarm.
       - `leave_one_video_out` ? the protocol that matters as more videos arrive:
         pick the config on all *other* videos, score on the held-out video. Reports
         per-video held-out scores + mean±std. This is the number to trust.

    Everything is segmenter-agnostic: callers pass a `predict_fn(config) ->
    List[Interval]` (e.g. WASB trajectory ? windows at those thresholds), so the same
    harness works for WASB, yolo_hybrid, or anything else. Pure + unit-tested; no I/O.
    """
    from __future__ import annotations

    import statistics as _st
    from typing import Callable, Dict, List, Sequence, Tuple, Any

    from backend.eval.metrics import score, Score, Interval

    Config = Any                              # opaque to this module (e.g. a thresholds
dict)
    PredictFn = Callable[[Config], List[Interval]]


    def pct_improvement(before: float, after: float) -> float:
        """Percentage change from `before` to `after` (e.g. 0.6 -> 0.75 == +25.0)."""
        if before <= 0:
            return float("inf") if after > 0 else 0.0
        return (after - before) / before * 100.0


    def evaluate(preds: List[Interval], gts: List[Interval], iou_threshold: float = 0.5) ->
```

```
Score:
42          """Precision / recall / F1 / mean-IoU of predicted vs ground-truth intervals."""
43          return score(preds, gts, iou_threshold)
44
45
46      def kfold_indices(n: int, k: int) -> List[Tuple[List[int], List[int]]]:
47          """Deterministic k-fold split of indices 0..n-1 into (train_idx, test_idx)."""
48          if n <= 0:
49              return []
50          k = max(2, min(k, n))                # need >=2 folds; cap at n
51          folds = [list(range(i, n, k)) for i in range(k)]   # round-robin ? balanced,
deterministic
52          splits = []
53          for f in range(k):
54              test = folds[f]
55              train = [i for i in range(n) if i not in set(test)]
56              if test and train:
57                  splits.append((train, test))
58          return splits
59
60
61      def select_best(predict_fn: PredictFn, configs: Sequence[Config], gts: List[Interval],
62                      metric: str = "f1", iou_threshold: float = 0.5) -> Tuple[Config, float]:
63          """Pick the config whose predictions maximise `metric` against `gts`."""
64          best_cfg, best_val = None, -1.0
65          for cfg in configs:
66              val = getattr(score(predict_fn(cfg), gts, iou_threshold), metric)
67              if val > best_val:
68                  best_cfg, best_val = cfg, val
69          return best_cfg, best_val
70
71
72      def cross_validate(predict_fn: PredictFn, configs: Sequence[Config], gts:
List[Interval],
73                         k: int = 5, metric: str = "recall", iou_threshold: float = 0.5) ->
Dict[str, Any]:
74          """k-fold threshold calibration on ONE video's rallies (within-video overfit guard).
75
76          Per fold: choose the best config on the train rallies, score it on the held-out
77          rallies. A large `generalization_gap` (train >> test) means the thresholds are
78          overfitting that video's quirks.
79
80          Default metric is **recall** on purpose: predictions are global (whole-video
81          windows), so scoring them against a *fold subset* of GT would unfairly count the
82          other rallies' windows as false positives ? precision/F1 are not well-defined
83          per-fold. Recall ("did the tuned thresholds still detect the held-out rallies?")
84          is the honest within-video generalization signal. For precision/F1, use
85          `leave_one_video_out` (each video scored whole) or `improvement_report` on a
86          held-out video.
87          """
88          splits = kfold_indices(len(gts), k)
89          if not splits:
90              return {"error": "not enough rallies for cross-validation", "n": len(gts)}
91          train_scores, test_scores, picked = [], [], []
92          for train_idx, test_idx in splits:
93              gts_tr = [gts[i] for i in train_idx]
94              gts_te = [gts[i] for i in test_idx]
95              cfg, tr = select_best(predict_fn, configs, gts_tr, metric, iou_threshold)
96              te = getattr(score(predict_fn(cfg), gts_te, iou_threshold), metric)
97              train_scores.append(tr)
98              test_scores.append(te)
99              picked.append(cfg)
100         mean_test = _st.mean(test_scores)
101         mean_train = _st.mean(train_scores)
102         return {
```

```
            "metric": metric,
            "folds": len(splits),
            "mean_test": mean_test,
            "std_test": _st.pstdev(test_scores) if len(test_scores) > 1 else 0.0,
            "mean_train": mean_train,
            "generalization_gap": mean_train - mean_test,   # >~0.1 = likely overfitting
            "picked_configs": picked,
        }


    def leave_one_video_out(videos: List[Dict[str, Any]], configs: Sequence[Config],
                            metric: str = "f1", iou_threshold: float = 0.5) -> Dict[str,
Any]:
        """Leave-one-video-out CV ? the honest metric as the corpus grows.

        `videos`: list of {"name": str, "predict_fn": PredictFn, "gts": [Interval]}.
        For each held-out video: pick the config that's best *pooled across the other
        videos*, then score it on the held-out one. The mean held-out score is the
        "precision on unseen videos" number; per-video spread shows stability.
        """
        if len(videos) < 2:
            return {"error": "need >=2 videos for leave-one-video-out", "n": len(videos)}
        per_video = []
        for i, held in enumerate(videos):
            others = [v for j, v in enumerate(videos) if j != i]

            def pooled_score(cfg: Config) -> float:
                vals = [getattr(score(v["predict_fn"](cfg), v["gts"], iou_threshold),
metric)
                        for v in others]
                return _st.mean(vals)

            best_cfg, best_train = max(((c, pooled_score(c)) for c in configs), key=lambda
t: t[1])
            held_score = getattr(score(held["predict_fn"](best_cfg), held["gts"],
iou_threshold), metric)
            per_video.append({"video": held.get("name", f"video_{i}"),
                              "held_out": held_score, "train_pooled": best_train, "config":
best_cfg})
        held_scores = [p["held_out"] for p in per_video]
        train_scores = [p["train_pooled"] for p in per_video]
        return {
            "metric": metric,
            "mean_held_out": _st.mean(held_scores),
            "std_held_out": _st.pstdev(held_scores) if len(held_scores) > 1 else 0.0,
            "generalization_gap": _st.mean(train_scores) - _st.mean(held_scores),
            "per_video": per_video,
        }


    def improvement_report(predict_fn: PredictFn, baseline_config: Config, tuned_config:
Config,
                           gts: List[Interval], iou_threshold: float = 0.5) -> Dict[str,
Any]:
        """The "+X% precision" headline: baseline vs tuned config on the same GT.

        NB: for an *honest* headline, pass GT the tuning never saw (a held-out video),
        or read the cross-validated `mean_test` from `cross_validate` instead.
        """
        b = score(predict_fn(baseline_config), gts, iou_threshold)
        t = score(predict_fn(tuned_config), gts, iou_threshold)
        return {
            "baseline": {"precision": b.precision, "recall": b.recall, "f1": b.f1,
"mean_iou": b.mean_iou},
            "tuned": {"precision": t.precision, "recall": t.recall, "f1": t.f1, "mean_iou":
```

```
t.mean_iou},
160             "precision_delta_pts": t.precision - b.precision,
161             "precision_delta_pct": pct_improvement(b.precision, t.precision),
162             "f1_delta_pct": pct_improvement(b.f1, t.f1),
163         }
164
```

6. user (2026-06-07T09:10:35.956Z)

```
1       """Per-video calibration driver: WASB trajectory + golden rallies -> tuned thresholds +
"+X%".
2
3       Wires the already-tested pieces together:
4         WASB trajectory CSV (Frame,Visibility,X,Y)
5           -> TrackNetRunner.trajectory_to_action_windows(cfg)   [predict_fn]
6           -> backend.eval.calibration (improvement_report / cross_validate)
7
8       Run (after a full-video WASB pass exists):
9           python -m backend.eval.calibrate_wasb --trajectory full_traj.csv \
10              --labels "golden_labels_badminton.csv" --fps 59.94 --frame-width 1920
11
12      Honesty note: the grid is *selected* on the full GT, so `improvement_report` on that
13      same GT is an **optimistic** upper bound. The trustworthy number is the
14      cross-validated held-out **recall** (and, once a 2nd labelled video exists,
15      `leave_one_video_out` for precision/F1 generalization).
16      """
17      from __future__ import annotations
18
19      import csv
20      import argparse
21      from typing import List, Tuple, Callable
22
23      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
24      from backend.eval.calibration import (
25          select_best, improvement_report, cross_validate, evaluate,
26      )
27
28      Interval = Tuple[float, float]
29      Config = Tuple[float, float, float]   # (velocity_thresh, merge_gap,
min_window_duration)
30
31      BASELINE: Config = (0.02, 2.0, 2.0)   # the WasbConfig / trajectory_to_action_windows
defaults
32
33
34      def load_golden_gts(csv_path: str) -> List[Interval]:
35          """Rally [start,end] seconds from the golden CSV (start_time/end_time cols)."""
36          gts: List[Interval] = []
37          with open(csv_path, newline="", encoding="utf-8") as f:
38              reader = csv.DictReader(f)
39              reader.fieldnames = [(n or "").strip().lower() for n in (reader.fieldnames or
[])]
40              for row in reader:
41                  row = {(k or "").strip().lower(): v for k, v in row.items()}
42                  try:
43                      s = float((row.get("start_time") or "").strip())
44                      e = float((row.get("end_time") or "").strip())
45                  except (TypeError, ValueError):
46                      continue
47                  if e > s:
48                      gts.append((s, e))
49          return sorted(gts)
50
51
52      def make_predict_fn(trajectory_csv: str, fps: float, frame_width: float) ->
```

```
Callable[[Config], List[Interval]]:
        """predict_fn(cfg) -> rally windows, from a cached WASB trajectory (parsed once)."""
        points = TrackNetRunner.parse_trajectory_csv(trajectory_csv)

        def predict_fn(cfg: Config) -> List[Interval]:
            vt, mg, mwd = cfg
            wins = TrackNetRunner.trajectory_to_action_windows(
                points, fps=fps, frame_width=frame_width,
                velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd,
            )
            return [(w["start"], w["end"]) for w in wins]

        return predict_fn


    def default_grid() -> List[Config]:
        return [(vt, mg, mwd)
                for vt in (0.005, 0.01, 0.02, 0.03, 0.05)
                for mg in (1.0, 2.0, 3.0)
                for mwd in (1.0, 2.0, 3.0)]


    def run(trajectory_csv: str, labels_csv: str, fps: float, frame_width: float,
            iou: float = 0.5) -> dict:
        gts = load_golden_gts(labels_csv)
        if not gts:
            raise SystemExit(f"no usable golden rallies in {labels_csv}")
        predict_fn = make_predict_fn(trajectory_csv, fps, frame_width)
        grid = default_grid()

        base = evaluate(predict_fn(BASELINE), gts, iou)
        best_cfg, best_f1 = select_best(predict_fn, grid, gts, metric="f1",
iou_threshold=iou)
        fit = improvement_report(predict_fn, BASELINE, best_cfg, gts, iou)   # OPTIMISTIC
(fit on full GT)
        cv = cross_validate(predict_fn, grid, gts, k=5)                      # HONEST held-
out recall

        print("=== WASB rally-segmentation calibration ===")
        print(f"golden rallies: {len(gts)}  | IoU thresh: {iou}")
        print(f"baseline cfg {BASELINE}: precision={base.precision:.3f}
recall={base.recall:.3f} "
              f"f1={base.f1:.3f} meanIoU={base.mean_iou:.3f}")
        print(f"tuned cfg    {best_cfg}: f1={best_f1:.3f}")
        print(f"  precision {fit['baseline']['precision']:.3f} ->
{fit['tuned']['precision']:.3f} "
              f"({fit['precision_delta_pct']:+.1f}% , fit-on-full = optimistic)")
        print(f"  f1        {fit['baseline']['f1']:.3f} -> {fit['tuned']['f1']:.3f} "
              f"({fit['f1_delta_pct']:+.1f}%)")
        print(f"HONEST (cross-val, {cv.get('folds','?')} folds): held-out recall "
              f"{cv.get('mean_test',float('nan')):.3f} ± {cv.get('std_test',0):.3f}, "
              f"generalization_gap {cv.get('generalization_gap',float('nan')):+.3f}")
        print("(For precision/F1 generalization, add a 2nd labelled video ->
leave_one_video_out.)")
        return {"baseline": base.as_dict(), "best_cfg": best_cfg, "fit": fit, "cv": cv}


    def main():
        ap = argparse.ArgumentParser()
        ap.add_argument("--trajectory", required=True, help="WASB trajectory CSV
(Frame,Visibility,X,Y)")
        ap.add_argument("--labels", required=True, help="golden labels CSV
(start_time/end_time cols)")
        ap.add_argument("--fps", type=float, default=59.94)
        ap.add_argument("--frame-width", type=float, default=1920.0)
```

```
109         ap.add_argument("--iou", type=float, default=0.5)
110         args = ap.parse_args()
111         run(args.trajectory, args.labels, args.fps, args.frame_width, args.iou)
112
113
114     if __name__ == "__main__":
115         main()
116
```

7. assistant (2026-06-07T09:10:43.081Z)

I'll continue reading the remaining files.

8. user (2026-06-07T09:10:44.026Z)

```
1       """Distill Gemini (teacher) into the LOCAL-ONLY pipeline (student) ? calibrate the
2       on-device WASB+heuristic knobs to match Gemini's rally labels, so production runs
3       100% local (no cloud, privacy lever intact, ~zero marginal cost).
4
5       Pipeline being tuned (all local, no Gemini at runtime):
6           WASB trajectory CSV -> trajectory_to_action_windows(velocity, merge_gap, min_dur)
7                                -> rally_gate (min_crossings)            -> rally windows
8
9       Ground truth = Gemini full-context silver labels (one CSV per video). With >=2
10      videos we use leave-one-video-out (tune on the others, score the held-out one) ?
11      the honest "how precise is the local pipeline on an unseen video" number. The
12      output is a single local config + the precision it buys, with NO Gemini dependency
13      shipped.
14
15      Reuses: TrackNetRunner windowing, rally_gate, calibrate_wasb.load_golden_gts,
16      calibration.{select_best, leave_one_video_out, improvement_report}.
17
18      Run:
19          python -m backend.eval.calibrate_local --manifest videos.json
20      where videos.json = [{"name","trajectory","fps","frame_width","labels"}, ...]
21      (labels = a Gemini full-context CSV; trajectory = the WASB CSV).
22      """
23      from __future__ import annotations
24
25      import json
26      import argparse
27      from typing import Callable, Dict, List, Tuple
28
29      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
30      from backend.pipeline.detectors import rally_gate
31      from backend.eval.calibrate_wasb import load_golden_gts
32      from backend.eval.calibration import select_best, leave_one_video_out,
improvement_report, evaluate
33
34      # (velocity_thresh, merge_gap, min_window_duration, min_crossings)
35      LocalConfig = Tuple[float, float, float, int]
36      BASELINE_LOCAL: LocalConfig = (0.02, 2.0, 2.0, 0)   # today's local default (windowing,
no gate)
37
38
39      def make_local_predict_fn(trajectory_csv: str, fps: float, frame_width: float) ->
Callable[[LocalConfig], list]:
40          """predict_fn(cfg) -> rally windows from a cached WASB trajectory, LOCAL ONLY
41          (windowing thresholds + the shuttle net-crossing gate). Parsed once."""
42          points = TrackNetRunner.parse_trajectory_csv(trajectory_csv)
43
44          def predict_fn(cfg: LocalConfig) -> List[Tuple[float, float]]:
45              vt, mg, mwd, mc = cfg
46              wins = TrackNetRunner.trajectory_to_action_windows(
47                  points, fps=fps, frame_width=frame_width,
```

```
                velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd,
            )
            wins = [(w["start"], w["end"]) for w in wins]
            if mc > 0:
                wins = rally_gate.apply_gate(points, wins, fps, min_crossings=mc)
            return wins

        return predict_fn


    def local_grid() -> List[LocalConfig]:
        return [(vt, mg, mwd, mc)
                for vt in (0.005, 0.01, 0.02, 0.03, 0.05)
                for mg in (1.0, 2.0, 3.0)
                for mwd in (1.0, 2.0, 3.0)
                for mc in (0, 1, 2, 3)]


    def build_videos(specs: List[Dict]) -> List[Dict]:
        videos = []
        for s in specs:
            pf = make_local_predict_fn(s["trajectory"], float(s["fps"]),
float(s["frame_width"]))
            gts = load_golden_gts(s["labels"])
            if not gts:
                raise SystemExit(f"no usable labels in {s['labels']} for {s.get('name')}")
            videos.append({"name": s.get("name", s["trajectory"]), "predict_fn": pf, "gts":
gts})
        return videos


    def run(specs: List[Dict], iou: float = 0.5, metric: str = "f1") -> dict:
        videos = build_videos(specs)
        grid = local_grid()
        print(f"=== Local-only calibration vs Gemini silver-GT ({len(videos)} videos, "
              f"{len(grid)} configs, IoU>={iou}, optimise {metric}) ===")

        # Per-video: today's baseline vs the best-fit local config (optimistic ? fit on that
video).
        per_video = []
        for v in videos:
            best_cfg, best_val = select_best(v["predict_fn"], grid, v["gts"], metric, iou)
            imp = improvement_report(v["predict_fn"], BASELINE_LOCAL, best_cfg, v["gts"],
iou)
            b, t = imp["baseline"], imp["tuned"]
            print(f"\n[{v['name']}] {len(v['gts'])} Gemini rallies")
            print(f"  baseline {BASELINE_LOCAL}: P={b['precision']:.3f} R={b['recall']:.3f}
F1={b['f1']:.3f}")
            print(f"  best-fit {best_cfg}: P={t['precision']:.3f} R={t['recall']:.3f}
F1={t['f1']:.3f} "
                  f"(F1 {imp['f1_delta_pct']:+.0f}%, fit-on-self = optimistic)")
            per_video.append({"name": v["name"], "best_cfg": best_cfg, "improvement": imp})

        # Honest cross-video: pick config on the OTHER videos, score the held-out one.
        lovo = leave_one_video_out(videos, grid, metric, iou)
        # Baseline's held-out score (no tuning) for the honest before/after.
        base_held = [evaluate(v["predict_fn"](BASELINE_LOCAL), v["gts"], iou) for v in
videos]
        base_mean = sum(getattr(s, metric) for s in base_held) / len(base_held)

        print("\n--- HONEST leave-one-video-out (the number to trust) ---")
        if "error" in lovo:
            print(f"  {lovo['error']} (need >=2 videos)")
        else:
            print(f"  baseline (untuned) mean held-out {metric}: {base_mean:.3f}")
```

```
106             print(f"  calibrated mean held-out {metric}:       {lovo['mean_held_out']:.3f}
"
107                   f"± {lovo['std_held_out']:.3f}  (gap {lovo['generalization_gap']:+.3f})")
108             for p in lovo["per_video"]:
109                 print(f"    held-out {p['video']}: {p['held_out']:.3f}  (cfg picked on
others: {p['config']})")
110         return {"per_video": per_video, "lovo": lovo, "baseline_held_out": base_mean,
"metric": metric}
111
112
113     def main() -> None:
114         ap = argparse.ArgumentParser(description="Calibrate the local-only pipeline against
Gemini silver labels.")
115         ap.add_argument("--manifest", required=True,
116                         help="JSON: [{name, trajectory, fps, frame_width, labels}, ...]")
117         ap.add_argument("--iou", type=float, default=0.5)
118         ap.add_argument("--metric", default="f1", choices=["f1", "precision", "recall"])
119         args = ap.parse_args()
120         with open(args.manifest, encoding="utf-8") as f:
121             specs = json.load(f)
122         run(specs, iou=args.iou, metric=args.metric)
123
124
125     if __name__ == "__main__":
126         main()
127
```

9. user (2026-06-07T09:10:44.496Z)

```
1       """Export tuned WASB rally segments for manual eyeball verification.
2
3       After calibration picks a tuned threshold config, this turns a cached WASB
4       trajectory into the actual rally [start,end] segments that config produces, so the
5       user can answer "are the rallies actually better?" ? by scrubbing the video to each
6       segment, by diffing tuned-vs-golden quantitatively, or by cutting clips.
7
8       Reuses (no new windowing/metrics/cutting logic):
9         - calibrate_wasb.make_predict_fn  -> trajectory -> segments for a given config
10        - eval.metrics.match_intervals/score -> tuned-vs-golden IoU comparison
11        - tools.rally_slicer.slice_rallies   -> optional clip cutting (the exported CSV is
12          written in the golden/slicer schema, so it feeds straight back in)
13
14      Run (uses the tuned config from the first calibration by default):
15          python -m backend.eval.export_wasb_segments --trajectory
~/clips/sample_long_traj.csv \
16              --labels "Annotation Setup/golden_labels_badminton.csv" --fps 59.94 --frame-
width 1920
17          # add --video <mp4> --slice  to also cut one padded clip per tuned rally
18      """
19      from __future__ import annotations
20
21      import csv
22      import os
23      import os.path as osp
24      import argparse
25      from typing import List, Tuple, Dict, Optional
26
27      from backend.eval.calibrate_wasb import make_predict_fn, load_golden_gts, BASELINE
28      from backend.eval.metrics import match_intervals, score
29      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
30      from backend.pipeline.detectors import rally_gate
31
32      Interval = Tuple[float, float]
33      Config = Tuple[float, float, float]
34
```

```
    # The tuned config from the first per-video calibration (velocity, merge_gap, min_dur);
    # F1 0.588 (baseline) -> 0.746. Override on the CLI to export a different operating
point.
    TUNED: Config = (0.05, 1.0, 1.0)

    SEG_FIELDS = ["rally_number", "start_time", "end_time", "duration", "start_mmss",
"end_mmss"]
    COMP_FIELDS = ["type", "ref", "start_time", "end_time", "mmss", "status",
                   "iou", "pred_start", "pred_end", "start_err_s", "end_err_s"]


    def seconds_to_mmss(t: float) -> str:
        """Human-scrubbable timestamp, e.g. 106.031 -> '1:46.031'."""
        t = max(0.0, float(t))
        m = int(t // 60)
        return f"{m}:{t - m * 60:06.3f}"


    def segments_to_rows(segments: List[Interval]) -> List[Dict[str, str]]:
        """Number segments 1..N in time order; emit the golden/slicer CSV schema."""
        rows = []
        for i, (s, e) in enumerate(segments, start=1):
            rows.append({
                "rally_number": str(i),
                "start_time": f"{s:.3f}",
                "end_time": f"{e:.3f}",
                "duration": f"{e - s:.3f}",
                "start_mmss": seconds_to_mmss(s),
                "end_mmss": seconds_to_mmss(e),
            })
        return rows


    def _ensure_parent(path: str) -> None:
        os.makedirs(osp.dirname(osp.abspath(path)) or ".", exist_ok=True)


    def write_segments_csv(path: str, segments: List[Interval]) -> str:
        """Write segments in the golden schema (rally_number/start_time/end_time + mm:ss).

        The schema is what tools.rally_slicer.load_rally_labels consumes, so the file
        doubles as input to clip cutting.
        """
        _ensure_parent(path)
        with open(path, "w", newline="", encoding="utf-8") as f:
            w = csv.DictWriter(f, fieldnames=SEG_FIELDS)
            w.writeheader()
            w.writerows(segments_to_rows(segments))
        return path


    def build_comparison(preds: List[Interval], gts: List[Interval],
                         iou_threshold: float = 0.5) -> dict:
        """Golden-centric comparison: per golden rally, did a tuned segment match (IoU),
        plus the boundary errors and any extra (false-positive) tuned segments."""
        mr = match_intervals(preds, gts, iou_threshold)
        sc = score(preds, gts, iou_threshold, match_result=mr)
        by_gt = {m.gt_index: m for m in mr.matches}

        rows: List[Dict[str, str]] = []
        for gi, (gs, ge) in enumerate(gts, start=0):
            m = by_gt.get(gi)
            if m is not None:
                ps, pe = preds[m.pred_index]
                rows.append({
```

```
                    "type": "golden", "ref": str(gi + 1),
                    "start_time": f"{gs:.3f}", "end_time": f"{ge:.3f}",
                    "mmss": f"{seconds_to_mmss(gs)}-{seconds_to_mmss(ge)}",
                    "status": "matched", "iou": f"{m.iou:.3f}",
                    "pred_start": f"{ps:.3f}", "pred_end": f"{pe:.3f}",
                    "start_err_s": f"{ps - gs:+.3f}", "end_err_s": f"{pe - ge:+.3f}",
                })
            else:
                rows.append({
                    "type": "golden", "ref": str(gi + 1),
                    "start_time": f"{gs:.3f}", "end_time": f"{ge:.3f}",
                    "mmss": f"{seconds_to_mmss(gs)}-{seconds_to_mmss(ge)}",
                    "status": "MISS", "iou": "0.000",
                    "pred_start": "", "pred_end": "", "start_err_s": "", "end_err_s": "",
                })
        for pi in mr.unmatched_pred_indices:                # extra tuned segments = false
positives
            ps, pe = preds[pi]
            rows.append({
                "type": "pred_extra", "ref": str(pi + 1),
                "start_time": f"{ps:.3f}", "end_time": f"{pe:.3f}",
                "mmss": f"{seconds_to_mmss(ps)}-{seconds_to_mmss(pe)}",
                "status": "false_positive", "iou": "",
                "pred_start": "", "pred_end": "", "start_err_s": "", "end_err_s": "",
            })
        return {"score": sc, "rows": rows}


    def write_comparison_csv(path: str, comparison: dict) -> str:
        _ensure_parent(path)
        with open(path, "w", newline="", encoding="utf-8") as f:
            w = csv.DictWriter(f, fieldnames=COMP_FIELDS)
            w.writeheader()
            w.writerows(comparison["rows"])
        return path


    def _with_suffix(path: str, suffix: str) -> str:
        root, ext = osp.splitext(path)
        return root + suffix + ext


    def run(trajectory_csv: str, fps: float, frame_width: float, cfg: Config = TUNED,
            out_csv: str = "output/wasb_tuned_segments.csv", labels_csv: Optional[str] =
None,
            iou: float = 0.5, baseline: bool = False,
            comparison_out: Optional[str] = None, min_crossings: int = 0) -> dict:
        predict_fn = make_predict_fn(trajectory_csv, fps, frame_width)
        # Gate A: drop between-point false fires (single service-feed tosses) by requiring
        # the shuttle to cross the net >= min_crossings times within the window.
        points = TrackNetRunner.parse_trajectory_csv(trajectory_csv) if min_crossings > 0
else None

        def _gate(segs: List[Interval], label: str) -> List[Interval]:
            if not points:
                return segs
            kept = rally_gate.apply_gate(points, segs, fps, min_crossings=min_crossings)
            print(f"[gate] {label}: {len(segs)} -> {len(kept)} segments "
                  f"(removed {len(segs) - len(kept)} with <{min_crossings} net-crossings)")
            return kept

        tuned_segs = _gate(predict_fn(cfg), "tuned")
        write_segments_csv(out_csv, tuned_segs)
        total = sum(e - s for s, e in tuned_segs)
        print("=== WASB tuned-segment export ===")
```

```
        print(f"tuned cfg {cfg}: {len(tuned_segs)} rally segments, {total:.1f}s of play ->
{out_csv}")
        result: dict = {"tuned_cfg": cfg, "num_tuned": len(tuned_segs), "out_csv": out_csv}

        if baseline:
            base_segs = _gate(predict_fn(BASELINE), "baseline")
            base_out = _with_suffix(out_csv, "_baseline")
            write_segments_csv(base_out, base_segs)
            print(f"baseline cfg {BASELINE}: {len(base_segs)} segments -> {base_out}")
            result["num_baseline"] = len(base_segs)
            result["baseline_csv"] = base_out

        if labels_csv:
            gts = load_golden_gts(labels_csv)
            if not gts:
                raise SystemExit(f"no usable golden rallies in {labels_csv}")
            comp = build_comparison(tuned_segs, gts, iou)
            sc = comp["score"]
            comp_out = comparison_out or _with_suffix(out_csv, "_vs_golden")
            write_comparison_csv(comp_out, comp)
            print(f"\n--- tuned vs golden ({len(gts)} rallies, IoU>={iou}) ---")
            print(f"matched (TP) {sc.true_positives} | misses (FN) {sc.false_negatives} | "
                  f"extra (FP) {sc.false_positives}")
            print(f"precision {sc.precision:.3f} | recall {sc.recall:.3f} | f1 {sc.f1:.3f} |
"
                  f"meanIoU {sc.mean_iou:.3f}")
            print(f"mean boundary error: start {sc.mean_start_error:.2f}s | end
{sc.mean_end_error:.2f}s")
            print(f"per-rally detail -> {comp_out}")
            result["comparison"] = sc.as_dict()
            result["comparison_csv"] = comp_out
        return result


    def _maybe_slice(video: str, segments_csv: str, clips_dir: str) -> None:
        """Cut one padded clip per exported rally (reuses the rally slicer)."""
        from backend.tools.rally_slicer import slice_rallies
        written = slice_rallies(video, segments_csv, ["all"], clips_dir)
        print(f"\ncut {len(written)} clip(s) -> {clips_dir}")


    def main() -> None:
        ap = argparse.ArgumentParser(description="Export tuned WASB rally segments for
eyeballing.")
        ap.add_argument("--trajectory", required=True, help="WASB trajectory CSV
(Frame,Visibility,X,Y)")
        ap.add_argument("--fps", type=float, default=59.94)
        ap.add_argument("--frame-width", type=float, default=1920.0)
        ap.add_argument("--velocity-thresh", type=float, default=TUNED[0])
        ap.add_argument("--merge-gap", type=float, default=TUNED[1])
        ap.add_argument("--min-duration", type=float, default=TUNED[2])
        ap.add_argument("--labels", default=None, help="golden labels CSV -> tuned-vs-golden
comparison")
        ap.add_argument("--iou", type=float, default=0.5)
        ap.add_argument("--out", default="output/wasb_tuned_segments.csv")
        ap.add_argument("--comparison-out", default=None)
        ap.add_argument("--baseline", action="store_true", help="also export baseline-config
segments")
        ap.add_argument("--min-crossings", type=int, default=0,
                        help="gate A: drop windows whose shuttle crosses the net < N times "
                             "(kills single-toss service-feed false fires; 0=off, 2
recommended)")
        ap.add_argument("--video", default=None, help="source video (with --slice) to cut
clips from")
        ap.add_argument("--slice", action="store_true", help="cut one padded clip per tuned
```

```
rally")
        ap.add_argument("--clips-dir", default="output/wasb_tuned_clips")
        args = ap.parse_args()

        cfg = (args.velocity_thresh, args.merge_gap, args.min_duration)
        res = run(args.trajectory, args.fps, args.frame_width, cfg=cfg, out_csv=args.out,
                  labels_csv=args.labels, iou=args.iou, baseline=args.baseline,
                  comparison_out=args.comparison_out, min_crossings=args.min_crossings)
        if args.slice:
            if not args.video:
                raise SystemExit("--slice requires --video")
            _maybe_slice(args.video, res["out_csv"], args.clips_dir)


    if __name__ == "__main__":
        main()

```

10. user (2026-06-07T09:10:45.436Z)

```
    """Batch evaluation across a collector export manifest.

    Phase 3 of the collector->indexer bridge: walk the `manifest.json` emitted by
    sports-data-collector's `curate export`, (optionally) process each video through
    a segmenter so detections land in the indexer DB, then score every video's
    detected rallies against its ground-truth CSV and aggregate the results.

    This module holds the *pure* pieces (path resolution, stage selection, metric
    aggregation) so they can be unit-tested without touching video files, the GPU,
    or any API. The orchestration/side-effects live in `run_phase3_eval.py`.
    """

    import os
    from typing import Dict, List, Optional


    # Segmenters that only ever populate the `final` play_segments table (no raw
    # candidate stage), so scoring `candidates` for them would be meaninglessly empty.
    _FINAL_ONLY_SEGMENTERS = {"motion", "gemini"}


    def resolve_video_path(local_path: Optional[str], videos_root: str) -> Optional[str]:
        """Resolve a manifest `local_path` to an absolute path.

        The collector stores paths like ``./data/videos\\shuttleset_1.mp4`` relative
        to its own repo root and with Windows separators. Normalise separators, strip
        a leading ``./``, and resolve relative paths against `videos_root` (the
        collector root). Returns None for a missing/empty path.
        """
        if not local_path:
            return None
        p = local_path.replace("\\", "/")
        if p.startswith("./"):
            p = p[2:]
        if os.path.isabs(p):
            return os.path.normpath(p)
        return os.path.normpath(os.path.join(videos_root, p))


    def default_stages_for(segmenter: str, requested: str = "auto") -> List[str]:
        """Pick which prediction stages to score.

        ``auto`` scores both stages for two-stage detectors (yolo_hybrid/tracknet_*)
        but only ``final`` for segmenters that don't emit candidates. An explicit
        request ('candidates' | 'final' | 'both') overrides the heuristic.
```

```
        """
        if requested == "both":
            return ["candidates", "final"]
        if requested in ("candidates", "final"):
            return [requested]
        # auto
        if segmenter in _FINAL_ONLY_SEGMENTERS:
            return ["final"]
        return ["candidates", "final"]


    def _safe_div(num: float, den: float) -> float:
        return num / den if den else 0.0


    def aggregate(per_video: List[dict], stages: List[str]) -> Dict[str, dict]:
        """Aggregate per-video stage scores into corpus-level metrics.

        `per_video` is a list of dicts shaped like ``report_to_dict`` output (each has
        ``stages[stage]["score"]`` with tp/fp/fn/mean_iou/...). For each stage we
        compute **micro** precision/recall/F1 (pooling TP/FP/FN across all videos) and
        a TP-weighted mean IoU, plus the macro mean of per-video F1.
        """
        out: Dict[str, dict] = {}
        for stage in stages:
            tp = fp = fn = 0
            iou_weighted = 0.0
            f1s: List[float] = []
            n_videos = 0
            for rep in per_video:
                sr = rep.get("stages", {}).get(stage)
                if not sr:
                    continue
                s = sr["score"]
                tp += s["true_positives"]
                fp += s["false_positives"]
                fn += s["false_negatives"]
                iou_weighted += s["mean_iou"] * s["true_positives"]
                f1s.append(s["f1"])
                n_videos += 1
            precision = _safe_div(tp, tp + fp)
            recall = _safe_div(tp, tp + fn)
            out[stage] = {
                "videos": n_videos,
                "tp": tp, "fp": fp, "fn": fn,
                "micro_precision": precision,
                "micro_recall": recall,
                "micro_f1": _safe_div(2 * precision * recall, precision + recall),
                "macro_f1": _safe_div(sum(f1s), len(f1s)),
                "mean_iou": _safe_div(iou_weighted, tp),
            }
        return out


    def format_aggregate(agg: Dict[str, dict]) -> str:
        """Human-readable corpus-level summary table."""
        lines = [
            "=" * 78,
            " CORPUS AGGREGATE",
            "=" * 78,
            f" {'Stage':<12} {'Vids':>4} {'TP':>5} {'FP':>5} {'FN':>5} "
            f"{'mPrec':>6} {'mRec':>6} {'microF1':>8} {'macroF1':>8} {'mIoU':>6}",
            "-" * 78,
        ]
        for stage, a in agg.items():
```

```
            lines.append(
                f" {stage:<12} {a['videos']:>4} {a['tp']:>5} {a['fp']:>5} {a['fn']:>5} "
                f"{a['micro_precision']:>6.2f} {a['micro_recall']:>6.2f} "
                f"{a['micro_f1']:>8.2f} {a['macro_f1']:>8.2f} {a['mean_iou']:>6.2f}"
            )
        lines.append("=" * 78)
        return "\n".join(lines)

```

11. user (2026-06-07T09:10:45.502Z)

```
"""Score the pipeline's detected rallies against ground-truth annotations.

Reads predictions straight from the SQLite DB (no pipeline re-run, no API cost),
so scoring is fast, deterministic, and free. Two prediction "stages" can be
scored:

  - ``candidates`` ? raw detector windows (``candidate_segments`` table). What
    the shuttle/person detector found, before AI confirmation.
  - ``final``      ? confirmed play segments (``play_segments`` table). After the
    AI handoff trimmed/validated boundaries.

Comparing the two tells you whether the weak link is *detection* (candidates
already miss rallies) or *segmentation* (candidates catch them but final is
wrong) ? see docs/EVALUATION.md.
"""

import logging
from dataclasses import dataclass, field
from typing import List, Dict, Optional

from backend.infrastructure.database import Database
from backend.eval import metrics
from backend.eval.metrics import Interval, Score, MatchResult

logger = logging.getLogger(__name__)

STAGES = ("candidates", "final")


@dataclass
class StageReport:
    stage: str
    pred_intervals: List[Interval]
    score: Score
    match_result: MatchResult
    pr_curve: List[Score] = field(default_factory=list)


@dataclass
class EvalReport:
    video_id: str
    iou_threshold: float
    gt_intervals: List[Interval]
    stages: Dict[str, StageReport] = field(default_factory=dict)


def _rows_to_intervals(rows: List[dict]) -> List[Interval]:
    """Pull (start_time, end_time) out of DB segment rows, sorted by start."""
    ivs = [(float(r["start_time"]), float(r["end_time"])) for r in rows]
    ivs.sort(key=lambda iv: iv[0])
    return ivs


def get_predictions(db: Database, video_id: str, stage: str,
```

```
                        detector: Optional[str] = None) -> List[Interval]:
        """Fetch predicted rally intervals for one stage from the DB."""
        if stage == "candidates":
            rows = db.query_candidate_segments(video_id, detector=detector)
        elif stage == "final":
            rows = db.query_play_segments(video_id, {})
        else:
            raise ValueError(f"unknown stage {stage!r}; expected one of {STAGES}")
        return _rows_to_intervals(rows)


    def evaluate(db: Database, video_id: str, gt_intervals: List[Interval],
                 stages: List[str], iou_threshold: float = 0.5,
                 detector: Optional[str] = None,
                 with_pr_curve: bool = False) -> EvalReport:
        """Score the requested stages for one video against ground truth."""
        report = EvalReport(video_id=video_id, iou_threshold=iou_threshold,
                            gt_intervals=gt_intervals)
        for stage in stages:
            preds = get_predictions(db, video_id, stage, detector=detector)
            mr = metrics.match_intervals(preds, gt_intervals, iou_threshold)
            s = metrics.score(preds, gt_intervals, iou_threshold, match_result=mr)
            curve = metrics.pr_curve(preds, gt_intervals) if with_pr_curve else []
            report.stages[stage] = StageReport(stage=stage, pred_intervals=preds,
                                               score=s, match_result=mr, pr_curve=curve)
        return report


    # ---------------------------------------------------------------------- reporting

    def _fmt_ts(seconds: float) -> str:
        seconds = max(0, int(seconds))
        h, rem = divmod(seconds, 3600)
        m, s = divmod(rem, 60)
        return f"{h:02d}:{m:02d}:{s:02d}"


    def report_to_dict(report: EvalReport) -> dict:
        """JSON-serialisable view of an EvalReport."""
        out = {
            "video_id": report.video_id,
            "iou_threshold": report.iou_threshold,
            "num_ground_truth": len(report.gt_intervals),
            "stages": {},
        }
        for stage, sr in report.stages.items():
            out["stages"][stage] = {
                "score": sr.score.as_dict(),
                "pr_curve": [s.as_dict() for s in sr.pr_curve],
            }
        return out


    def format_report_text(report: EvalReport, show_failures: bool = False) -> str:
        """Human-readable console report for one video."""
        lines = [
            "=" * 64,
            f" Rally evaluation ? video '{report.video_id}'",
            f" Ground-truth rallies: {len(report.gt_intervals)} | IoU threshold:
{report.iou_threshold}",
            "=" * 64,
            f" {'Stage':<12} {'Pred':>5} {'TP':>4} {'FP':>4} {'FN':>4} "
            f"{'Prec':>6} {'Rec':>6} {'F1':>6} {'mIoU':>6} {'dStart':>7} {'dEnd':>7}",
            "-" * 64,
        ]
```

```
119         for stage in report.stages:
120             s = report.stages[stage].score
121             lines.append(
122                 f" {stage:<12} {s.num_pred:>5} {s.true_positives:>4} {s.false_positives:>4}
"
123                 f"{s.false_negatives:>4} {s.precision:>6.2f} {s.recall:>6.2f} {s.f1:>6.2f} "
124                 f"{s.mean_iou:>6.2f} {s.mean_start_error:>6.1f}s {s.mean_end_error:>6.1f}s"
125             )
126         lines.append("=" * 64)
127 
128         if show_failures:
129             for stage, sr in report.stages.items():
130                 misses = [report.gt_intervals[i] for i in
sr.match_result.unmatched_gt_indices]
131                 fps = [sr.pred_intervals[i] for i in sr.match_result.unmatched_pred_indices]
132                 lines.append(f"\n [{stage}] missed rallies (false negatives):
{len(misses)}")
133                 for iv in misses:
134                     lines.append(f"   - {_fmt_ts(iv[0])}?{_fmt_ts(iv[1])}")
135                 lines.append(f" [{stage}] spurious detections (false positives):
{len(fps)}")
136                 for iv in fps:
137                     lines.append(f"   - {_fmt_ts(iv[0])}?{_fmt_ts(iv[1])}")
138 
139         return "\n".join(lines)
140 
```

12. assistant (2026-06-07T09:10:49.174Z)

Now let me read the remaining files.

13. user (2026-06-07T09:10:50.123Z)

```
1       """Build a training set for the offline, API-free learned rally segmenter.
2 
3       Phase 4 of the collector->indexer bridge (see docs/ROADMAP.md, ADR-004). The idea:
4       a detector (YOLO player-motion today, optionally WASB shuttle-motion later)
5       produces candidate rally windows with features; the collector's ground-truth
6       rally intervals tell us which windows are real. A lightweight classifier learns
7       that mapping, replacing the paid Gemini refinement with a local model.
8 
9       This module is the *data layer* ? pure and unit-tested:
10 
11        - `label_candidates`  ? generic: label windows positive/negative by whether they
12          match a ground-truth rally (same IoU matching the evaluator uses, so training
13          labels are consistent with how we score).
14        - `extract_yolo_features` ? YOLO-specific feature vector from a candidate row's
15          persisted `stats`. Pluggable: a WASB substrate supplies its own feature_fn.
16        - `build_training_set` ? glue: rows + GT -> (X, y, intervals).
17 
18      No model training or I/O here; that lives in the trainer/segmenter once the
19      substrate is chosen from the Phase 3 candidate-recall result.
20      """
21 
22      from typing import Callable, Dict, List, Tuple
23 
24      from backend.eval.metrics import Interval, match_intervals
25 
26      # YOLO Stage-1 persists these per candidate window in `candidate_segments.stats`
27      # (see HybridYoloSegmenter). Duration is derived from the window bounds.
28      YOLO_FEATURE_NAMES = [
29          "duration", "peak_velocity", "mean_velocity", "active_frame_count",
"track_id_count",
30      ]
31 
```

```

    def extract_yolo_features(row: Dict) -> List[float]:
        """Feature vector for one YOLO candidate row (dict from query_candidate_segments).

        `stats` is already JSON-deserialized by the DB layer. Missing fields default
        to 0.0 so a sparse/old row still yields a well-formed vector.
        """
        stats = row.get("stats") or {}
        duration = float(row["end_time"]) - float(row["start_time"])
        return [
            duration,
            float(stats.get("peak_velocity", 0.0)),
            float(stats.get("mean_velocity", 0.0)),
            float(stats.get("active_frame_count", 0)),
            float(stats.get("track_id_count", 0)),
        ]


    def label_candidates(candidate_intervals: List[Interval], gt_intervals: List[Interval],
                         iou_threshold: float = 0.5) -> List[int]:
        """Label each candidate window 1 (real rally) or 0 (spurious).

        A window is positive iff it is matched one-to-one to a ground-truth rally at
        `iou_threshold` ? the *same* matching the evaluator uses for true positives,
        so the classifier is trained on exactly what we score. Substrate-agnostic:
        works for YOLO or WASB-derived windows alike.
        """
        mr = match_intervals(candidate_intervals, gt_intervals, iou_threshold)
        positive = {m.pred_index for m in mr.matches}
        return [1 if i in positive else 0 for i in range(len(candidate_intervals))]


    def build_training_set(
        candidate_rows: List[Dict],
        gt_intervals: List[Interval],
        iou_threshold: float = 0.5,
        feature_fn: Callable[[Dict], List[float]] = extract_yolo_features,
    ) -> Tuple[List[List[float]], List[int], List[Interval]]:
        """Turn one video's candidate rows + ground truth into (X, y, intervals).

        `feature_fn` is pluggable so a WASB substrate can supply shuttle-motion
        features without changing the labeling logic. Returns the feature matrix `X`,
        binary labels `y`, and the candidate intervals (handy for downstream merging).
        """
        intervals: List[Interval] = [
            (float(r["start_time"]), float(r["end_time"])) for r in candidate_rows
        ]
        y = label_candidates(intervals, gt_intervals, iou_threshold)
        X = [feature_fn(r) for r in candidate_rows]
        return X, y, intervals

```

14. user (2026-06-07T09:10:50.596Z)

```

    """A tiny, dependency-free logistic-regression classifier (numpy only).

    The offline learned segmenter (ADR-004) must run locally with no external API and
    no heavy ML stack ? the project already depends on numpy, and sklearn is not
    installed. This implements just enough: standardized features, balanced-class
    gradient descent, sigmoid probabilities, and JSON (de)serialization so a trained
    model is a small, human-readable, version-controllable file.

    It deliberately stays simple: the rally-window features are few and roughly
    linearly separable (high player motion + sustained activity => real rally), so a
    well-regularized linear model is a sensible, inspectable baseline. Swap in a
```

```
    richer model later behind the same fit/predict_proba/save/load interface.
    """

    import json
    from typing import List, Optional

    import numpy as np


    class LogisticRegression:
        def __init__(self, lr: float = 0.1, epochs: int = 1000, l2: float = 1e-3):
            self.lr = lr
            self.epochs = epochs
            self.l2 = l2
            self.w: Optional[np.ndarray] = None
            self.b: float = 0.0
            self.mean: Optional[np.ndarray] = None
            self.std: Optional[np.ndarray] = None
            self.feature_names: Optional[List[str]] = None

        @staticmethod
        def _sigmoid(z: np.ndarray) -> np.ndarray:
            # Clip for numerical stability (avoid overflow in exp).
            return 1.0 / (1.0 + np.exp(-np.clip(z, -30, 30)))

        def _standardize(self, X: np.ndarray) -> np.ndarray:
            return (X - self.mean) / self.std

        def fit(self, X, y, feature_names: Optional[List[str]] = None,
                class_weight: str = "balanced") -> "LogisticRegression":
            """Fit with optional balanced class weighting (rally windows are imbalanced
            ? many spurious candidates per real rally)."""
            X = np.asarray(X, dtype=float)
            y = np.asarray(y, dtype=float)
            if X.ndim != 2 or X.shape[0] == 0:
                raise ValueError("X must be a non-empty 2D array")
            self.feature_names = feature_names

            self.mean = X.mean(axis=0)
            self.std = X.std(axis=0)
            self.std[self.std == 0] = 1.0  # guard constant features
            Xs = self._standardize(X)
            n, d = Xs.shape

            if class_weight == "balanced":
                n_pos = max(1, int(y.sum()))
                n_neg = max(1, int((1 - y).sum()))
                w_pos, w_neg = n / (2.0 * n_pos), n / (2.0 * n_neg)
            else:
                w_pos = w_neg = 1.0
            sw = np.where(y == 1, w_pos, w_neg)

            self.w = np.zeros(d)
            self.b = 0.0
            for _ in range(self.epochs):
                p = self._sigmoid(Xs @ self.w + self.b)
                err = (p - y) * sw
                grad_w = Xs.T @ err / n + self.l2 * self.w
                grad_b = float(err.mean())
                self.w -= self.lr * grad_w
                self.b -= self.lr * grad_b
            return self

        def predict_proba(self, X) -> np.ndarray:
            if self.w is None:
```

```
                raise RuntimeError("model is not fitted")
            Xs = self._standardize(np.asarray(X, dtype=float))
            return self._sigmoid(Xs @ self.w + self.b)

        def predict(self, X, threshold: float = 0.5) -> np.ndarray:
            return (self.predict_proba(X) >= threshold).astype(int)

        # --- persistence (small JSON, no pickle/joblib) --------------------------

        def to_dict(self) -> dict:
            return {
                "model": "logreg",
                "lr": self.lr, "epochs": self.epochs, "l2": self.l2,
                "w": self.w.tolist() if self.w is not None else None,
                "b": self.b,
                "mean": self.mean.tolist() if self.mean is not None else None,
                "std": self.std.tolist() if self.std is not None else None,
                "feature_names": self.feature_names,
            }

        @classmethod
        def from_dict(cls, d: dict) -> "LogisticRegression":
            m = cls(lr=d.get("lr", 0.1), epochs=d.get("epochs", 1000), l2=d.get("l2", 1e-3))
            m.w = np.asarray(d["w"], dtype=float) if d.get("w") is not None else None
            m.b = float(d.get("b", 0.0))
            m.mean = np.asarray(d["mean"], dtype=float) if d.get("mean") is not None else
None
            m.std = np.asarray(d["std"], dtype=float) if d.get("std") is not None else None
            m.feature_names = d.get("feature_names")
            return m

        def save(self, path: str) -> None:
            with open(path, "w") as f:
                json.dump(self.to_dict(), f, indent=2)

        @classmethod
        def load(cls, path: str) -> "LogisticRegression":
            with open(path) as f:
                return cls.from_dict(json.load(f))

```

15. user (2026-06-07T09:10:51.074Z)

```
    """Regression runner ? verify the production tool against golden-set ground truth.

    Implements the explicit ask: *"given a video, get a golden set created and verify
    its precision and recall against the production tool."* This is deliberately a thin
    wrapper over machinery that already exists:

        ground truth  ? AnnotationProvider (file / vendor / oracle)  [backend.annotations]
        predictions   ? the production pipeline's stored DB rows     [backend.eval.harness]
        comparison    ? interval matching + P/R/F1                   [backend.eval.metrics]
        baseline      ? a per-video snapshot of "known good" numbers [this module]

    A "regression" is a precision/recall drop beyond `tolerance` versus the snapshotted
    baseline. Tiers map to annotation providers: e.g. tier "file" ? FileProvider (CI),
    later tier "vendor" ? HumanVendorProvider (authoritative), tier "auto" ? an
    independent-lineage oracle (fast smoke alarm). See docs/GOLDEN_SET_IMPLEMENTATION.md.
    """
    import os
    import json
    import logging
    from dataclasses import dataclass, asdict
    from typing import Dict, List, Optional

```

```
    from backend.infrastructure.database import Database
    from backend.eval import metrics
    from backend.eval.harness import get_predictions
    from backend.annotations import annotation_registry

    logger = logging.getLogger(__name__)

    DEFAULT_BASELINE_PATH = os.path.join("output", "regression_baseline.json")
    DEFAULT_TOLERANCE = 0.05


    @dataclass
    class RegressionResult:
        video_id: str
        stage: str
        iou_threshold: float
        precision: float
        recall: float
        f1: float
        # Baseline values compared against (None when no baseline existed yet).
        baseline_precision: Optional[float]
        baseline_recall: Optional[float]
        tolerance: float
        regressed: bool
        reasons: List[str]

        def as_dict(self) -> dict:
            return asdict(self)


    # ---------------------------------------------------------------- baseline store

    def load_baselines(path: str = DEFAULT_BASELINE_PATH) -> Dict[str, dict]:
        """Load the per-video baseline snapshot file (returns {} if absent)."""
        if not os.path.isfile(path):
            return {}
        with open(path) as f:
            return json.load(f)


    def _baseline_key(video_id: str, stage: str) -> str:
        return f"{video_id}::{stage}"


    def save_baseline(video_id: str, stage: str, precision: float, recall: float,
                      f1: float, path: str = DEFAULT_BASELINE_PATH) -> None:
        """Snapshot a video/stage's current numbers as the new 'known good' baseline.

        Use this to (re)baseline after a *legitimate* production improvement, so old
        baselines don't raise false regressions. Stored as plain JSON for easy diffing.
        """
        baselines = load_baselines(path)
        baselines[_baseline_key(video_id, stage)] = {
            "video_id": video_id, "stage": stage,
            "precision": precision, "recall": recall, "f1": f1,
        }
        os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
        with open(path, "w") as f:
            json.dump(baselines, f, indent=2, sort_keys=True)
        logger.info("Saved baseline for %s (precision=%.3f recall=%.3f)",
_baseline_key(video_id, stage), precision, recall)


    # ---------------------------------------------------------------- the runner

```

```
    def regress(db: Database, video_id: str, ground_truth: List[metrics.Interval],
                stage: str = "final", iou_threshold: float = 0.5,
                detector: Optional[str] = None,
                tolerance: float = DEFAULT_TOLERANCE,
                baseline_path: str = DEFAULT_BASELINE_PATH) -> RegressionResult:
        """Score stored predictions for one video against ground truth and compare to
baseline.

        `ground_truth` is a list of (start, end) intervals ? typically obtained from an
        AnnotationProvider (see `regress_with_provider`). A regression is flagged when
        precision OR recall falls more than `tolerance` below the snapshotted baseline.
        If no baseline exists yet, `regressed` is False (nothing to compare to) and the
        caller can choose to snapshot the current numbers via `save_baseline`.
        """
        preds = get_predictions(db, video_id, stage, detector=detector)
        s = metrics.score(preds, ground_truth, iou_threshold)

        baselines = load_baselines(baseline_path)
        b = baselines.get(_baseline_key(video_id, stage))

        reasons: List[str] = []
        regressed = False
        base_p = base_r = None
        if b is not None:
            base_p, base_r = b["precision"], b["recall"]
            if s.precision < base_p - tolerance:
                regressed = True
                reasons.append(f"precision {s.precision:.3f} < baseline {base_p:.3f} -
{tolerance}")
            if s.recall < base_r - tolerance:
                regressed = True
                reasons.append(f"recall {s.recall:.3f} < baseline {base_r:.3f} -
{tolerance}")
        else:
            reasons.append("no baseline recorded for this video/stage ? nothing to compare")

        return RegressionResult(
            video_id=video_id, stage=stage, iou_threshold=iou_threshold,
            precision=s.precision, recall=s.recall, f1=s.f1,
            baseline_precision=base_p, baseline_recall=base_r,
            tolerance=tolerance, regressed=regressed, reasons=reasons,
        )


    def regress_with_provider(db: Database, video_id: str, video_ref: str,
                              tier: str = "file", stage: str = "final",
                              iou_threshold: float = 0.5,
                              detector: Optional[str] = None,
                              tolerance: float = DEFAULT_TOLERANCE,
                              baseline_path: str = DEFAULT_BASELINE_PATH,
                              guideline: Optional[str] = None,
                              sport: str = "badminton",
                              provider_kwargs: Optional[dict] = None) -> RegressionResult:
        """End-to-end: obtain ground truth from the `tier` annotation provider, then
regress.

        `tier` is an annotation-provider registry key ("file" today; "vendor"/"auto"
        once those providers land). `video_ref` is whatever that provider expects (a CSV
        path for FileProvider, a video_id/URL for a vendor). `video_id` keys the stored
        predictions in the DB (the file MD5).
        """
        provider = annotation_registry.get(tier, **(provider_kwargs or {}))
        gt = provider.annotate(video_ref, guideline=guideline, sport=sport)
        return regress(db, video_id, gt, stage=stage, iou_threshold=iou_threshold,
                       detector=detector, tolerance=tolerance, baseline_path=baseline_path)
```

```
148
149
150     def format_result(r: RegressionResult) -> str:
151         verdict = "REGRESSED" if r.regressed else "ok"
152         head = (f"[{verdict}] {r.video_id} ({r.stage}) "
153                 f"precision={r.precision:.3f} recall={r.recall:.3f} f1={r.f1:.3f}")
154         if r.baseline_precision is not None:
155             head += f" | baseline p={r.baseline_precision:.3f} r={r.baseline_recall:.3f}
tol={r.tolerance}"
156         return head + "".join(f"\n   - {why}" for why in r.reasons)
157
```

16. user (2026-06-07T09:10:52.001Z)

```
1       """Distill Gemini (teacher) into a LEARNED local rally classifier (student).
2
3       Threshold-tuning the local knobs hit a low ceiling (see calibrate_local). This goes
4       further: learn "is this WASB candidate a real rally?" from Gemini silver labels,
5       using a few resolution/fps-INVARIANT local features, so the model transfers across
6       videos and runs 100% local at inference (no Gemini, privacy lever intact).
7
8       Flow (all local at runtime):
9         WASB trajectory -> recall-oriented candidate windows (propose many; classifier
10        filters) -> per-candidate features [duration, peak_velocity, mean_velocity,
11        active_ratio, net_crossings] -> LogisticRegression(prob >= threshold) -> merge
12        -> rally windows.
13
14      Training labels come from Gemini full-context labels via the SAME IoU matching the
15      evaluator uses (training.label_candidates). Honest leave-one-video-out: train on the
16      other videos' candidates, predict the held-out one, score vs its Gemini labels, and
17      compare against the threshold baseline. Reuses classifier.py, training.label_candidates,
18      rally_gate, metrics, calibrate_wasb.load_golden_gts, gemini_refine.merge_overlaps.
19
20      Run:
21          python -m backend.eval.distill_local --manifest videos.json --model-out
output/local_rally_model.json
22      """
23      from __future__ import annotations
24
25      import json
26      import argparse
27      from typing import Dict, List, Tuple
28
29      import numpy as np
30
31      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
32      from backend.pipeline.detectors import rally_gate
33      from backend.eval.calibrate_wasb import load_golden_gts
34      from backend.eval.training import label_candidates
35      from backend.eval.classifier import LogisticRegression
36      from backend.eval.metrics import score
37      from backend.eval.gemini_refine import merge_overlaps
38
39      Interval = Tuple[float, float]
40
41      FEATURE_NAMES = ["duration", "peak_velocity", "mean_velocity", "active_ratio",
"net_crossings"]
42      # Recall-oriented proposal: deliberately permissive so real rallies are among the
43      # candidates (the classifier removes the junk). (velocity_thresh, merge_gap, min_dur)
44      PROPOSAL = (0.005, 1.0, 0.5)
45      BASELINE_LOCAL = (0.02, 2.0, 2.0)   # today's production-ish windowing, no learning
46
47
48      def build_candidates(trajectory_csv: str, fps: float, frame_width: float,
49                           proposal: Tuple[float, float, float] = PROPOSAL) ->
```

```
Tuple[List[Interval], List[List[float]]]:
        """WASB trajectory -> (candidate intervals, fps/resolution-invariant feature
rows)."""
        points = TrackNetRunner.parse_trajectory_csv(trajectory_csv)
        vt, mg, mwd = proposal
        wins = TrackNetRunner.trajectory_to_action_windows(
            points, fps=fps, frame_width=frame_width,
            velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd)
        intervals = [(w["start"], w["end"]) for w in wins]
        gated = rally_gate.gate_windows(points, intervals, fps, min_crossings=0)  # for net-
crossings
        X: List[List[float]] = []
        for w, g in zip(wins, gated):
            dur = max(1e-6, w["end"] - w["start"])
            win_frames = max(1.0, dur * fps)
            X.append([
                dur,                                            # rally length (sec) ?
fps-invariant
                float(w["peak_velocity"]),                      # already normalized by
frame_width
                float(w["mean_velocity"]),
                float(w["active_frame_count"]) / win_frames,    # active-shuttle ratio ?
fps-invariant
                float(g.crossings),                             # net crossings (count) ?
the rally signal
            ])
        return intervals, X


    def baseline_windows(trajectory_csv: str, fps: float, frame_width: float) ->
List[Interval]:
        points = TrackNetRunner.parse_trajectory_csv(trajectory_csv)
        vt, mg, mwd = BASELINE_LOCAL
        wins = TrackNetRunner.trajectory_to_action_windows(
            points, fps=fps, frame_width=frame_width,
            velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd)
        return [(w["start"], w["end"]) for w in wins]


    def load_video(spec: Dict, iou: float = 0.5) -> Dict:
        fps, fw = float(spec["fps"]), float(spec["frame_width"])
        intervals, X = build_candidates(spec["trajectory"], fps, fw)
        gts = load_golden_gts(spec["labels"])
        y = label_candidates(intervals, gts, iou)
        return {"name": spec.get("name", spec["trajectory"]), "intervals": intervals, "X":
X,
                "y": y, "gts": gts, "baseline": baseline_windows(spec["trajectory"], fps,
fw)}


    def predict_rallies(model: LogisticRegression, X: List[List[float]], intervals:
List[Interval],
                        threshold: float = 0.5) -> List[Interval]:
        if not intervals:
            return []
        proba = model.predict_proba(X)
        pos = [iv for iv, p in zip(intervals, proba) if p >= threshold]
        return merge_overlaps(pos, gap_tol=1.0)


    def run(specs: List[Dict], iou: float = 0.5, threshold: float = 0.5, model_out: str =
None) -> dict:
        videos = [load_video(s, iou) for s in specs]
        print(f"=== Distilled local rally classifier vs Gemini silver-GT "
              f"({len(videos)} videos, IoU>={iou}, prob>={threshold}) ===")
```

```
        for v in videos:
            ceil = score(v["intervals"], v["gts"], iou).recall
            print(f"[{v['name']}] {len(v['intervals'])} candidates ({sum(v['y'])} pos) | "
                  f"proposal recall ceiling {ceil:.3f} | {len(v['gts'])} Gemini rallies")

        result: dict = {"videos": [v["name"] for v in videos]}
        if len(videos) >= 2:
            print("\n--- HONEST leave-one-video-out: learned vs threshold baseline (held-
out) ---")
            clf, base = [], []
            for i, held in enumerate(videos):
                others = [v for j, v in enumerate(videos) if j != i]
                X = [row for v in others for row in v["X"]]
                y = [t for v in others for t in v["y"]]
                model = LogisticRegression().fit(X, y, FEATURE_NAMES)
                preds = predict_rallies(model, held["X"], held["intervals"], threshold)
                sc = score(preds, held["gts"], iou)
                bsc = score(held["baseline"], held["gts"], iou)
                clf.append(sc)
                base.append(bsc)
                print(f"  held-out {held['name']}: LEARNED P={sc.precision:.3f}
R={sc.recall:.3f} F1={sc.f1:.3f}"
                      f"  |  baseline P={bsc.precision:.3f} R={bsc.recall:.3f}
F1={bsc.f1:.3f}")
            mean = lambda xs, a: sum(getattr(s, a) for s in xs) / len(xs)
            print(f"  MEAN held-out: LEARNED P={mean(clf,'precision'):.3f}
R={mean(clf,'recall'):.3f} "
                  f"F1={mean(clf,'f1'):.3f}  |  baseline P={mean(base,'precision'):.3f} "
                  f"R={mean(base,'recall'):.3f} F1={mean(base,'f1'):.3f}")
            result["mean_learned_f1"] = mean(clf, "f1")
            result["mean_baseline_f1"] = mean(base, "f1")

        # Final model on ALL videos (for production / running on a fresh held-out video).
        Xall = [row for v in videos for row in v["X"]]
        yall = [t for v in videos for t in v["y"]]
        final = LogisticRegression().fit(Xall, yall, FEATURE_NAMES)
        weights = dict(zip(FEATURE_NAMES, [round(float(w), 3) for w in final.w]))
        print(f"\nfinal model standardized weights (sign/size = influence): {weights}")
        if model_out:
            final.save(model_out)
            print(f"final model (all videos) -> {model_out}")
            result["model_out"] = model_out
        return result


    def main() -> None:
        ap = argparse.ArgumentParser(description="Distill Gemini labels into a local rally
classifier.")
        ap.add_argument("--manifest", required=True,
                        help="JSON: [{name, trajectory, fps, frame_width, labels}, ...]
(labels = Gemini CSV)")
        ap.add_argument("--iou", type=float, default=0.5)
        ap.add_argument("--threshold", type=float, default=0.5, help="classifier probability
cutoff")
        ap.add_argument("--model-out", default="output/local_rally_model.json")
        args = ap.parse_args()
        with open(args.manifest, encoding="utf-8") as f:
            specs = json.load(f)
        run(specs, iou=args.iou, threshold=args.threshold, model_out=args.model_out)


    if __name__ == "__main__":
        main()

```

17. user (2026-06-07T09:10:52.485Z)

```
1       """Refine WASB rally candidates with Gemini (verify boundaries + reject false fires).
2
3       WASB (shuttle-only) over-fires on between-point service feeds. This drives the
4       EXISTING Gemini provider over the SHORT WASB candidate clips (not the whole video):
5       each candidate window is cut to a small padded clip and handed to Gemini, which ?
6       seeing the actual footage ? returns the true rally inside it (tight boundaries) or
7       nothing (it was a toss / dead time). Cheap because uploads are seconds-long clips,
8       not GB-scale chunks; high-precision because Gemini judges each candidate visually.
9
10      Reuses (no reinvented wheels): providers.GeminiProvider (via provider_registry),
11      sports.badminton prompt (sport_registry), utils.video.extract_video_chunk,
12      calibrate_wasb.make_predict_fn, export_wasb_segments.write_segments_csv.
13
14      This mirrors wasb_hybrid's Phase-3 handoff but runs standalone on a cached
15      trajectory (no WASB re-run, no DB) ? an eval/tuning driver, not the live pipeline.
16
17      Run (GEMINI_API_KEY in env):
18          python -m backend.eval.gemini_refine --video Adarsh.MP4 --trajectory adarsh_traj.csv
\
19              --fps 59.94 --frame-width 1920 --out output/adarsh_gemini.csv [--limit 4]
20      """
21      from __future__ import annotations
22
23      import os
24      import os.path as osp
25      import time
26      import argparse
27      import concurrent.futures
28      from typing import List, Tuple, Optional
29
30      from backend.eval.calibrate_wasb import make_predict_fn, BASELINE
31      from backend.eval.export_wasb_segments import TUNED, write_segments_csv, seconds_to_mmss
32      from backend.utils.video import get_video_duration, extract_video_chunk
33      from backend.sports import sport_registry
34      from backend.providers import provider_registry
35
36      Interval = Tuple[float, float]
37
38
39      def merge_overlaps(intervals: List[Interval], gap_tol: float = 0.0) -> List[Interval]:
40          """Union of overlapping intervals, optionally stitching across a small gap.
41
42          gap_tol > 0 merges two intervals separated by <= gap_tol seconds ? used to heal
brief
43          shuttle-invisibility splits WITHIN one rally, while keeping genuine between-rally
gaps
44          (several seconds) as separate rallies. (Replaces the old envelope-per-candidate
logic,
45          which over-merged multiple real rallies WASB had bundled into one candidate.)"""
46          merged: List[Interval] = []
47          for s, e in sorted(intervals):
48              if merged and s <= merged[-1][1] + gap_tol:
49                  merged[-1] = (merged[-1][0], max(merged[-1][1], e))
50              else:
51                  merged.append((s, e))
52          return merged
53
54
55      def _offset_segments(raw: list, clip_start: float, clip_end: float) -> List[Interval]:
56          """Map a Gemini clip's returned segments back to full-video seconds, clamped to
clip."""
57          out: List[Interval] = []
58          for sg in raw or []:
```

```
            try:
                st = float(sg["start_time"]) + clip_start
                en = float(sg["end_time"]) + clip_start
            except (TypeError, ValueError, KeyError):
                continue
            st = max(clip_start, st)
            en = min(clip_end, en)
            if en > st:
                out.append((round(st, 3), round(en, 3)))
        return out


    def refine_window(api_key: str, model: str, sport_profile, video: str, window: Interval,
                      pad: float, duration: float, tmpdir: str, idx: int,
                      clip_scale_width: Optional[int] = 1280) -> Tuple[int, Interval,
List[Interval]]:
        # Own provider/client per call: the genai client isn't reliably thread-safe, and a
        # shared one across workers throws socket errors. A fresh lazy client is cheap.
        provider = provider_registry.get("gemini", api_key=api_key, model_name=model)
        s, e = window
        cs = max(0.0, s - pad)
        ce = min(duration, e + pad) if duration > 0 else e + pad
        clip = osp.join(tmpdir, f"cand_{idx:04d}.mp4")
        # Downscale the uploaded clip (Gemini analyses at low res) -> small, fast, under the
        # 500MB upload cap even for 4K/5K sources.
        if not extract_video_chunk(video, cs, ce - cs, clip, reencode=True,
scale_width=clip_scale_width):
            return idx, window, []
        try:
            prompt = sport_profile.get_prompt(chunk_duration=ce - cs)
            raw, _ = provider.analyze_video(clip, prompt, chunk_duration=ce - cs,
label=f"cand{idx}")
            mapped = _offset_segments(raw, cs, ce)
        finally:
            try:
                os.remove(clip)
            except OSError:
                pass
        # Keep Gemini's segments separate ? a WASB candidate can bundle several real rallies
        # (merge_gap bridged the between-point gaps), and collapsing them to one envelope
swept
        # dead time into a "rally" (precision loss). Tiny invisibility splits are healed
later
        # by the gap-tolerant global merge in run().
        return idx, window, mapped


    def run(video: str, trajectory: str, fps: float, frame_width: float,
            cfg=BASELINE, pad: float = 3.0, model: str = "gemini-flash-latest",
            max_workers: int = 4, out_csv: str = "output/gemini_refined.csv",
            min_dur: float = 2.0, limit: int = 0, clip_scale_width: Optional[int] = 1280) ->
dict:
        api_key = os.environ.get("GEMINI_API_KEY")
        if not api_key:
            raise SystemExit("GEMINI_API_KEY not set in env")
        sport_profile = sport_registry.get("badminton")
        duration = get_video_duration(video)

        candidates = make_predict_fn(trajectory, fps, frame_width)(cfg)
        if limit:
            candidates = candidates[:limit]
        tmpdir = osp.join("output", "gemini_refine_clips")
        os.makedirs(tmpdir, exist_ok=True)

        print(f"[gemini_refine] {len(candidates)} WASB candidates (cfg {cfg}) -> Gemini
```

```
{model}, "
               f"pad {pad}s, {max_workers} workers", flush=True)
        t0 = time.time()
        per_candidate = []
        with concurrent.futures.ThreadPoolExecutor(max_workers=max_workers) as ex:
            futs = [ex.submit(refine_window, api_key, model, sport_profile, video, w, pad,
duration,
                              tmpdir, i, clip_scale_width)
                    for i, w in enumerate(candidates)]
            for fut in concurrent.futures.as_completed(futs):
                idx, window, mapped = fut.result()
                per_candidate.append((idx, window, mapped))
                tag = "REJECT (no rally)" if not mapped else \
                      " ".join(f"{seconds_to_mmss(a)}-{seconds_to_mmss(b)}" for a, b in
mapped)
                print(f"  cand {idx:>2}
{seconds_to_mmss(window[0])}-{seconds_to_mmss(window[1])} -> {tag}", flush=True)

        raw_intervals = [iv for _, _, m in per_candidate for iv in m]
        refined = [(s, e) for s, e in merge_overlaps(raw_intervals, gap_tol=1.0) if e - s >=
min_dur]
        write_segments_csv(out_csv, refined)
        try:
            os.rmdir(tmpdir)
        except OSError:
            pass

        kept_cands = sum(1 for _, _, m in per_candidate if m)
        play = sum(e - s for s, e in refined)
        print(f"\n[gemini_refine] {kept_cands}/{len(candidates)} candidates had a rally; "
              f"{len(refined)} merged rallies, {play:.1f}s play -> {out_csv} "
              f"({time.time() - t0:.0f}s)", flush=True)
        return {"refined": refined, "num_candidates": len(candidates),
                "candidates_with_rally": kept_cands, "out_csv": out_csv}


    def full_video_rallies(video: str, model: str = "gemini-flash-latest",
                           out_csv: str = "output/gemini_fullvideo.csv", clip_scale_width:
Optional[int] = 1280,
                           min_dur: float = 2.0, chunk_sec: float = 120.0, overlap: float =
10.0,
                           max_workers: int = 3) -> List[Interval]:
        """Full-context pass: hand Gemini the WHOLE clip (downscaled, chunked to stay under
the
        upload cap), so it finds every rally itself ? not bounded by WASB's candidate
recall.
        Independent of the trajectory; complements refine()."""
        api_key = os.environ.get("GEMINI_API_KEY")
        if not api_key:
            raise SystemExit("GEMINI_API_KEY not set in env")
        sport_profile = sport_registry.get("badminton")
        duration = get_video_duration(video)
        starts, t = [], 0.0
        while t < duration:
            starts.append(t)
            t += max(1.0, chunk_sec - overlap)
        tmpdir = osp.join("output", "gemini_full_clips")
        os.makedirs(tmpdir, exist_ok=True)

        def do_chunk(ci: int, cstart: float) -> List[Interval]:
            cend = min(cstart + chunk_sec, duration)
            clip = osp.join(tmpdir, f"chunk_{ci:03d}.mp4")
            if not extract_video_chunk(video, cstart, cend - cstart, clip, reencode=True,
scale_width=clip_scale_width):
                return []
```

```
            try:
                provider = provider_registry.get("gemini", api_key=api_key,
model_name=model)
                prompt = sport_profile.get_prompt(chunk_duration=cend - cstart)
                raw, _ = provider.analyze_video(clip, prompt, chunk_duration=cend - cstart,
label=f"chunk{ci}")
                return _offset_segments(raw, cstart, cend)
            finally:
                try:
                    os.remove(clip)
                except OSError:
                    pass

        print(f"[gemini_full] {len(starts)} chunk(s) of {video} (<= {chunk_sec}s, width
{clip_scale_width}) "
              f"-> Gemini {model}", flush=True)
        with concurrent.futures.ThreadPoolExecutor(max_workers=max_workers) as ex:
            chunks = [f.result() for f in [ex.submit(do_chunk, i, s) for i, s in
enumerate(starts)]]
        rallies = [(s, e) for s, e in merge_overlaps([iv for c in chunks for iv in c],
gap_tol=1.0) if e - s >= min_dur]
        write_segments_csv(out_csv, rallies)
        try:
            os.rmdir(tmpdir)
        except OSError:
            pass
        play = sum(e - s for s, e in rallies)
        print(f"[gemini_full] {len(rallies)} rallies, {play:.1f}s play -> {out_csv}",
flush=True)
        return rallies


    def main() -> None:
        ap = argparse.ArgumentParser(description="Refine WASB candidates with Gemini.")
        ap.add_argument("--video", required=True)
        ap.add_argument("--trajectory", default=None, help="WASB trajectory CSV (required
unless --full-video)")
        ap.add_argument("--fps", type=float, default=59.94)
        ap.add_argument("--frame-width", type=float, default=1920.0)
        ap.add_argument("--config", choices=["baseline", "tuned"], default="baseline",
                        help="WASB candidate config to feed Gemini (baseline = whole
rallies, good recall)")
        ap.add_argument("--pad", type=float, default=3.0)
        ap.add_argument("--model", default="gemini-flash-latest")
        ap.add_argument("--max-workers", type=int, default=4)
        ap.add_argument("--min-dur", type=float, default=2.0)
        ap.add_argument("--out", default="output/gemini_refined.csv")
        ap.add_argument("--limit", type=int, default=0, help="only the first N candidates
(smoke test)")
        ap.add_argument("--clip-scale-width", type=int, default=1280,
                        help="downscale uploaded clips to this width (0=keep source res)")
        ap.add_argument("--full-video", action="store_true",
                        help="full-context pass: send the whole clip (chunked/downscaled) to
Gemini "
                             "instead of refining WASB candidates (needs no --trajectory)")
        args = ap.parse_args()
        scale = args.clip_scale_width or None
        if args.full_video:
            full_video_rallies(args.video, model=args.model, out_csv=args.out,
                               clip_scale_width=scale, min_dur=args.min_dur,
max_workers=args.max_workers)
            return
        if not args.trajectory:
            raise SystemExit("--trajectory is required unless --full-video")
        cfg = BASELINE if args.config == "baseline" else TUNED
```

```
        run(args.video, args.trajectory, args.fps, args.frame_width, cfg=cfg, pad=args.pad,
            model=args.model, max_workers=args.max_workers, out_csv=args.out,
            min_dur=args.min_dur, limit=args.limit, clip_scale_width=scale)


    if __name__ == "__main__":
        main()

```

18. user (2026-06-07T09:10:52.832Z)

```
    """E1 ? audio GO/NO-GO probe (docs/AUDIO_RALLY_DETECTION_PLAN.md).

    Answers, on REAL audio: does the shuttle "thwack" survive well enough that hit
    peaks (a) cluster inside rallies far more than dead time, (b) recover rallies the
    WASB windows miss (our recall bottleneck), and (c) form usable audio-only rally
    proposals? No training, no fusion ? purely diagnostic, so we know whether to build
    the rest of the audio path.

    Reuses: audio.hit_detector, calibrate_wasb.load_golden_gts/make_predict_fn,
    eval.metrics. Labels may be human-gold OR Gemini-silver ? silver is fine for the
    SNR/efficacy question E1 asks.
    """
    from __future__ import annotations

    import argparse
    from typing import List, Tuple

    from backend.pipeline.audio.hit_detector import detect_hits_in_video, HitEvent
    from backend.eval.calibrate_wasb import load_golden_gts, make_predict_fn, BASELINE
    from backend.eval.metrics import match_intervals, score
    from backend.utils.video import get_video_duration

    Interval = Tuple[float, float]


    def cluster_hits(hits: List[HitEvent], gap_sec: float = 2.0, min_hits: int = 2,
                     pad: float = 0.3) -> List[Interval]:
        """Group hits whose inter-hit gap <= gap_sec into rally proposals; keep clusters
        with >= min_hits (a lone hit = a service feed, not a rally)."""
        ts = sorted(h.t for h in hits)
        if not ts:
            return []
        out: List[Interval] = []
        start = last = ts[0]
        cnt = 1
        for t in ts[1:]:
            if t - last <= gap_sec:
                last, cnt = t, cnt + 1
            else:
                if cnt >= min_hits:
                    out.append((max(0.0, start - pad), last + pad))
                start = last = t
                cnt = 1
        if cnt >= min_hits:
            out.append((max(0.0, start - pad), last + pad))
        return out


    def _hits_in(hits: List[HitEvent], s: float, e: float) -> int:
        return sum(1 for h in hits if s <= h.t <= e)


    def run(video: str, labels_csv: str, trajectory: str, fps: float, frame_width: float,
            k: float = 2.5, cluster_gap: float = 2.0, min_hits: int = 2, iou: float = 0.5)
```

```
-> dict:
55          hits = detect_hits_in_video(video, k=k)
56          gts = load_golden_gts(labels_csv)
57          wasb = make_predict_fn(trajectory, fps, frame_width)(BASELINE)
58          dur = get_video_duration(video) or (max((e for _, e in gts + wasb), default=0.0))
59
60          print(f"=== E1 audio probe: {video} ===")
61          print(f"  {len(hits)} hits detected (k={k}) over {dur:.0f}s =
{len(hits)/max(1,dur)*60:.1f} hits/min | "
62                f"{len(gts)} labeled rallies, {len(wasb)} WASB baseline windows")
63
64          # (1) Discrimination: hit density inside rallies vs dead time.
65          rally_dur = sum(e - s for s, e in gts)
66          dead_dur = max(1e-6, dur - rally_dur)
67          hits_in = sum(_hits_in(hits, s, e) for s, e in gts)
68          hits_out = len(hits) - hits_in
69          dens_in = hits_in / max(1e-6, rally_dur)
70          dens_out = hits_out / dead_dur
71          ratio = dens_in / dens_out if dens_out > 0 else float("inf")
72          rallies_with_hits = sum(1 for s, e in gts if _hits_in(hits, s, e) >= 2)
73          print(f"  [discrimination] in-rally {dens_in:.2f} hits/s vs dead-time {dens_out:.2f}
hits/s "
74                f"? ratio {ratio:.1f}x | rallies with >=2 hits: {rallies_with_hits}/{len(gts)}
"
75                f"({100*rallies_with_hits/max(1,len(gts)):.0f}%)")
76
77          # (2) Recall recovery: of rallies WASB MISSES, how many have an audio hit-cluster?
78          mr = match_intervals(wasb, gts, iou)
79          missed = [gts[i] for i in mr.unmatched_gt_indices]
80          recoverable = [r for r in missed if _hits_in(hits, *r) >= min_hits]
81          rec_rate = len(recoverable) / len(missed) if missed else 0.0
82          print(f"  [recall recovery] WASB misses {len(missed)}/{len(gts)} rallies; "
83                f"audio hit-clusters sit in {len(recoverable)} of them ? {100*rec_rate:.0f}%
recoverable")
84
85          # (3) Audio-only rally proposals vs labels.
86          props = cluster_hits(hits, cluster_gap, min_hits)
87          sc = score(props, gts, iou)
88          print(f"  [audio-only proposals] {len(props)} rallies | P={sc.precision:.3f}
R={sc.recall:.3f} "
89                f"F1={sc.f1:.3f} (vs WASB-only F1 {score(wasb, gts, iou).f1:.3f})")
90          return {
91              "video": video, "k": k, "n_hits": len(hits), "duration": dur,
92              "discrimination_ratio": ratio, "rallies_with_hits_pct": 100 * rallies_with_hits
/ max(1, len(gts)),
93              "wasb_missed": len(missed), "recall_recoverable_pct": 100 * rec_rate,
94              "audio_only": sc.as_dict(), "wasb_only_f1": score(wasb, gts, iou).f1,
95          }
96
97
98      def main() -> None:
99          ap = argparse.ArgumentParser(description="E1 audio go/no-go probe.")
100         ap.add_argument("--video", required=True)
101         ap.add_argument("--labels", required=True, help="golden or Gemini-silver rally CSV")
102         ap.add_argument("--trajectory", required=True, help="WASB trajectory CSV")
103         ap.add_argument("--fps", type=float, default=59.94)
104         ap.add_argument("--frame-width", type=float, default=1920.0)
105         ap.add_argument("--k", type=float, default=2.5, help="onset sensitivity (lower =
more hits)")
106         ap.add_argument("--cluster-gap", type=float, default=2.0)
107         ap.add_argument("--min-hits", type=int, default=2)
108         ap.add_argument("--iou", type=float, default=0.5)
109         args = ap.parse_args()
110         run(args.video, args.labels, args.trajectory, args.fps, args.frame_width,
111             k=args.k, cluster_gap=args.cluster_gap, min_hits=args.min_hits, iou=args.iou)
```

```
112
113
114     if __name__ == "__main__":
115         main()
116
```

19. assistant (2026-06-07T09:10:55.967Z)

All 14 files are read. Now let me check the tests directory to map tests to this subsystem, and verify the exact repo-relative paths.

20. user (2026-06-07T09:10:56.548Z)

```
backend\eval\__init__.py
backend\eval\annotations.py
backend\eval\batch.py
backend\eval\classifier.py
backend\eval\metrics.py
backend\eval\training.py
backend\eval\calibration.py
backend\eval\gemini_refine.py
backend\eval\audio\__init__.py
backend\eval\audio\e1_probe.py
backend\eval\calibrate_local.py
backend\eval\calibrate_wasb.py
backend\eval\distill_local.py
backend\eval\export_wasb_segments.py
backend\eval\harness.py
backend\eval\regression.py
```

21. user (2026-06-07T09:10:56.716Z)

```
tests\__init__.py
tests\test_eval_annotations.py
tests\test_eval_batch.py
tests\test_eval_classifier.py
tests\test_eval_metrics.py
tests\test_eval_training.py
tests\test_geometry.py
tests\test_providers.py
tests\test_sport_profiles.py
tests\test_video.py
tests\test_annotation_providers.py
tests\test_rally_slicer.py
tests\test_calibration.py
tests\test_calibrate_wasb.py
tests\test_export_wasb_segments.py
tests\test_gemini_refine.py
tests\test_provider_gemini.py
tests\test_calibrate_local.py
tests\test_distill_local.py
tests\test_automated_provider.py
tests\test_base.py
tests\test_config.py
tests\test_database.py
tests\test_eval_harness.py
tests\test_eval_regression.py
tests\test_gemini.py
tests\test_hit_detector.py
tests\test_rally_gate.py
tests\test_reporting_harvest.py
tests\test_tracknet_hybrid.py
tests\test_tracknet_runner.py
tests\test_validators_base.py
```

tests\test_wasb_cache.py
tests\test_yolo_hybrid.py
tests\test_reporting_spec_faq.py

22. assistant (2026-06-07T09:12:16.426Z)

I have all the information needed. Here is the dense, anchor-rich CODE_MAP chunk for the Eval & calibration subsystem.

Eval & calibration (`backend/eval/`)

Segmenter-agnostic measurement, threshold calibration, Gemini-distillation, and regression-guard layer. Pure-metrics core (no I/O) feeds CLI drivers that turn cached WASB trajectories + golden/silver labels into tuned configs, exported segments, learned classifiers, and "+X%" headlines. Legend: §=data contract, ?=registry/extension point, ?=footgun.

Pure metrics core (no I/O, deterministic, unit-tested)
- `@backend/eval/metrics.py` ? interval-matching primitives. `interval_iou(a,b)` temporal IoU (0.0 if degenerate/non-overlapping); `match_intervals(preds,gts,iou_threshold=0.5)` greedy **one-to-one** matching by descending IoU; `score(preds,gts,iou_threshold=0.5,match_result=None)` ? P/R/F1 + `mean_iou` + boundary errors; `pr_curve(preds,gts,thresholds=None)` sweeps 0.1..0.9.
  - §`Interval = Tuple[float,float]` ? `(start,end)` seconds, **the shared currency of the whole subsystem**.
  - §`@backend/eval/metrics.py::Match` (`pred_index,gt_index,iou`); §`MatchResult` (`matches`, `unmatched_pred_indices`=FPs, `unmatched_gt_indices`=FNs); §`Score` (dataclass: `iou_threshold,num_pred,num_gt,true_positives,false_positives,false_negatives,precision,recall,f1,mean_iou,mean_start_error,mean_end_error`; `.as_dict()`).
  - ? matching is greedy (highest-IoU-first), not optimal/Hungarian ? ties broken by `(pi,gi)` for determinism. ? `score` with empty matches returns 0.0 for `mean_iou`/boundary errors, not NaN.
- `@backend/eval/annotations.py` ? ground-truth CSV loader. `parse_timestamp(value)` accepts decimal sec OR `MM:SS`/`HH:MM:SS` (mixed OK), raises `ValueError` on unparseable; `load_annotations(csv_path)` ? sorted intervals; `write_scaffold(csv_path)` writes header-only file.
  - §annotation CSV: `start,end` header optional/auto-skipped; `#` comments + blanks ignored; ? raises (not skips) on malformed row or `start >= end` ? labeling mistakes fail loudly. ? this `start,end` schema differs from the golden `start_time/end_time` schema in `calibrate_wasb.load_golden_gts`.
- `@backend/eval/calibration.py` ? overfit-guarded improvement measurement (pure, segmenter-agnostic via `predict_fn`). `pct_improvement(before,after)`; `evaluate(preds,gts,iou=0.5)`; `kfold_indices(n,k)` deterministic round-robin split; `select_best(predict_fn,configs,gts,metric="f1",iou=0.5)`; `cross_validate(...,k=5,metric="recall")` within-video k-fold ? `generalization_gap`; `leave_one_video_out(videos,configs,metric="f1")` ? `mean_held_out`; `improvement_report(predict_fn,baseline_config,tuned_config,gts)`.
  - §`Config = Any` (opaque); §`PredictFn = Callable[[Config], List[Interval]]` ? **the plug-in seam** all drivers conform to. ? any segmenter (WASB/yolo_hybrid/learned) participates by supplying a `predict_fn`.
  - §`leave_one_video_out` videos contract: `{"name":str, "predict_fn":PredictFn, "gts":[Interval]}`.
  - ? `cross_validate` default metric is **recall** on purpose ? predictions are whole-video, so precision/F1 are ill-defined per-fold (other rallies' windows count as FPs). ? `generalization_gap > ~0.1` = likely overfit. ? `pct_improvement` returns `inf` when `before<=0` and `after>0`.

Calibration / export CLI drivers (I/O + side effects; thin glue over the core)
- `@backend/eval/calibrate_wasb.py` ? per-video WASB threshold calibration driver. `run(trajectory_csv,labels_csv,fps,frame_width,iou=0.5)`; `make_predict_fn(trajectory_csv,fps,frame_width)` parses trajectory ONCE then closes over `TrackNetRunner.trajectory_to_action_windows`; `load_golden_gts(csv_path)` (§golden CSV: `start_time`/`end_time` cols, case/space-normalized, silently skips unparseable rows);

`default_grid()` 5×3×3=45 configs; `main()` argparse entrypoint.
  - §`Config = (velocity_thresh, merge_gap, min_window_duration)`; const `BASELINE=(0.02,2.0,2.0)`. **`load_golden_gts` + `make_predict_fn` + `BASELINE` are imported by 4 downstream drivers** (calibrate_local, export_wasb_segments, gemini_refine, audio/e1_probe).
  - ? grid is selected on full GT ? `improvement_report` here is an **optimistic upper bound**; trustworthy number is the cross-val held-out recall.
- `@backend/eval/calibrate_local.py` ? distill Gemini?local knobs (windowing + net-crossing gate, no cloud at runtime). `make_local_predict_fn(...)` adds `rally_gate.apply_gate` when `min_crossings>0`; `local_grid()` 5×3×3×4=180 configs; `build_videos(specs)`; `run(specs,iou=0.5,metric="f1")` reports per-video best-fit (optimistic) + `leave_one_video_out` (honest); `main()` `--manifest`.
  - §`LocalConfig = (velocity_thresh,merge_gap,min_window_duration,min_crossings)`; const `BASELINE_LOCAL=(0.02,2.0,2.0,0)`.
  - §manifest JSON: `[{name,trajectory,fps,frame_width,labels}, ...]` (`labels`=Gemini silver CSV) ? **shared with `distill_local`**.
  - ? `build_videos` raises `SystemExit` if any labels file yields no rallies.
- `@backend/eval/export_wasb_segments.py` ? turn a tuned config into eyeball-able segments + golden comparison + optional clip cut.
`run(trajectory_csv,fps,frame_width,cfg=TUNED,...,min_crossings=0)`; `segments_to_rows`/`write_segments_csv` (§golden/slicer schema `SEG_FIELDS=[rally_number,start_time,end_time,duration,start_mmss,end_mmss]`); `build_comparison(preds,gts,iou=0.5)` golden-centric per-rally matched/MISS/false_positive rows; `write_comparison_csv` (§`COMP_FIELDS`); `seconds_to_mmss(t)`; `_maybe_slice` lazy-imports `backend.tools.rally_slicer.slice_rallies`; `main()`.
  - §`Config=(velocity,merge_gap,min_dur)`; const `TUNED=(0.05,1.0,1.0)` (F1 0.588?0.746). ? `write_segments_csv` output is consumed by `rally_slicer.load_rally_labels` AND re-imported by `gemini_refine` ? schema is load-bearing across modules. ? inner `_gate` re-parses trajectory only when `min_crossings>0`.

DB-backed scoring + corpus aggregation + regression

- `@backend/eval/harness.py` ? score stored DB predictions (no pipeline re-run, free/deterministic). `get_predictions(db,video_id,stage,detector=None)` reads `candidates`?`query_candidate_segments` / `final`?`query_play_segments`; `evaluate(db,video_id,gt_intervals,stages,iou=0.5,detector=None,with_pr_curve=False)`; `report_to_dict`; `format_report_text(report,show_failures=False)`; `_rows_to_intervals`, `_fmt_ts`.
  - §`STAGES=("candidates","final")` ? candidates=raw detector windows, final=AI-confirmed; diffing the two localizes detection-vs-segmentation failure. §`StageReport`(stage,pred_intervals,score,match_result,pr_curve); §`EvalReport`(video_id,iou_threshold,gt_intervals,stages). ? `get_predictions` raises `ValueError` on unknown stage. **`get_predictions` is reused by `regression.py`.**
- `@backend/eval/batch.py` ? pure corpus-aggregation pieces (testable without GPU/DB/API; orchestration lives in `run_phase3_eval.py`, not here). `resolve_video_path(local_path,videos_root)` normalizes collector `./data\...` Windows paths; `default_stages_for(segmenter,requested="auto")`; `aggregate(per_video,stages)` micro P/R/F1 (pooled TP/FP/FN) + TP-weighted mean IoU + macro-F1; `format_aggregate(agg)`.
  - ? §`_FINAL_ONLY_SEGMENTERS={"motion","gemini"}` ? segmenters with no candidate stage; `auto` scores both stages for two-stage detectors (yolo_hybrid/tracknet_*) else `final` only. §`aggregate` consumes `report_to_dict`-shaped dicts (`stages[stage]["score"]`). ? `_safe_div` returns 0.0 on zero denominator (no div-by-zero).
- `@backend/eval/regression.py` ? regression runner vs snapshotted "known-good" baseline. `regress(db,video_id,ground_truth,stage="final",iou=0.5,...,tolerance=0.05,baseline_path)`; `regress_with_provider(db,video_id,video_ref,tier="file",...)` pulls GT from ?`annotation_registry.get(tier)` (`backend.annotations`); `load_baselines`/`save_baseline`; `format_result`.
  - §`RegressionResult` dataclass (video_id,stage,iou_threshold,precision,recall,f1,baseline_precision,baseline_recall,tolerance,regressed,reasons; `.as_dict()`). §baseline store: JSON keyed `"{video_id}::{stage}"`, `DEFAULT_BASELINE_PATH=output/regression_baseline.json`, `DEFAULT_TOLERANCE=0.05`.
  - ? `tier`?provider mapping: "file"?FileProvider (CI), "vendor"?HumanVendorProvider, "auto"?oracle. ? no baseline ? `regressed=False` (nothing to compare). ? regression flagged when **precision OR recall** drops > tolerance (F1 not gated). ? re-baseline only after a *legitimate* improvement or you suppress real regressions.

Learned local segmenter (ADR-004: offline, API-free)

- `@backend/eval/training.py` ? pure data layer for the learned classifier. `label_candidates(candidate_intervals,gt_intervals,iou=0.5)` labels via the **same `match_intervals`** the evaluator uses (train/score consistency); `extract_yolo_features(row)`; `build_training_set(candidate_rows,gt_intervals,iou=0.5,feature_fn=extract_yolo_features)` ? `(X,y,intervals)`.
  - ? `feature_fn` is the substrate plug-in seam (YOLO today, WASB later supplies its own). §`YOLO_FEATURE_NAMES=[duration,peak_velocity,mean_velocity,active_frame_count,track_id_count]` read from `candidate_segments.stats` (JSON, populated by `HybridYoloSegmenter`). ? missing stats fields default 0.0 (sparse/old rows still well-formed).
- `@backend/eval/classifier.py` ? dependency-free `LogisticRegression` (numpy only, no sklearn). `fit(X,y,feature_names=None,class_weight="balanced")` standardizes + balanced-class GD; `predict_proba`/`predict(threshold=0.5)`; `to_dict`/`from_dict`/`save`/`load` (small human-readable JSON, no pickle). Ctor: `lr=0.1,epochs=1000,l2=1e-3`.
  - ? `_sigmoid` clips z to [-30,30]; constant features guarded (`std==0 ? 1.0`). ? stores feature standardization (`mean`/`std`) in the model ? inference MUST use same feature order. §JSON contract has `"model":"logreg"` tag.
- `@backend/eval/distill_local.py` ? distill Gemini ? a **learned** classifier (goes beyond `calibrate_local`'s threshold ceiling). `build_candidates(...)` recall-oriented proposal + fps/resolution-INVARIANT features; `baseline_windows`; `load_video(spec,iou=0.5)`; `predict_rallies(model,X,intervals,threshold=0.5)` filters by proba then `gemini_refine.merge_overlaps(gap_tol=1.0)`; `run(specs,iou,threshold,model_out)` leave-one-video-out learned-vs-baseline + final all-videos model; `main()`.
  - §`FEATURE_NAMES=[duration,peak_velocity,mean_velocity,active_ratio,net_crossings]` (deliberately fps/res-invariant for cross-video transfer); consts `PROPOSAL=(0.005,1.0,0.5)` (permissive recall-first), `BASELINE_LOCAL=(0.02,2.0,2.0)`. ? `net_crossings` via `rally_gate.gate_windows(...,min_crossings=0)` (`.crossings` count). ? leave-one-video-out only runs with ?2 videos; final model always trained on all.

Gemini refinement driver (eval/tuning, standalone ? not the live pipeline)

- `@backend/eval/gemini_refine.py` ? refine WASB candidates with Gemini over SHORT padded clips (precision lever; over-fires on service feeds otherwise). `merge_overlaps(intervals,gap_tol=0.0)` (gap_tol heals shuttle-invisibility splits without merging real rallies ? **reused by `distill_local`**); `refine_window(...)` cuts+downscales clip, fresh provider per call; `run(video,trajectory,...,cfg=BASELINE,pad=3.0,model="gemini-flash-latest",max_workers=4,...)`; `full_video_rallies(...)` full-context chunked pass (recall not bounded by WASB candidates); `_offset_segments`; `main()` (`--full-video` skips `--trajectory`).
  - ? `provider_registry.get("gemini",...)` + `sport_registry.get("badminton")`. ? **fresh genai client per worker** ? shared client across threads throws socket errors (not thread-safe). ? keeps Gemini's per-clip segments separate (collapsing to one envelope swept dead time into "rallies" = precision loss); tiny splits healed by the global `gap_tol=1.0` merge. ? requires `GEMINI_API_KEY` env (raises `SystemExit`). ? imports `BASELINE` from calibrate_wasb and `TUNED`/`write_segments_csv`/`seconds_to_mmss` from export_wasb_segments.

Audio probe (diagnostic only)

- `@backend/eval/audio/e1_probe.py` ? E1 GO/NO-GO audio probe (docs/AUDIO_RALLY_DETECTION_PLAN.md); no training/fusion, purely diagnostic. `cluster_hits(hits,gap_sec=2.0,min_hits=2,pad=0.3)` groups thwacks into rally proposals; `run(video,labels_csv,trajectory,fps,frame_width,k=2.5,...)` reports (1) discrimination (in-rally vs dead-time hit density ratio), (2) recall recovery of WASB-missed rallies, (3) audio-only proposal P/R/F1; `_hits_in`; `main()`.
  - reuses `pipeline.audio.hit_detector.detect_hits_in_video/HitEvent` + `calibrate_wasb.{load_golden_gts,make_predict_fn,BASELINE}` + `metrics.{match_intervals,score}`. ? outcome (per MEMORY/audio-e1-findings): recall 100% but discrimination only ~2.2×, band-pass null ? **audio NOT adopted for fusion** (PR #35). ? `lone hit = service feed`, so `min_hits>=2` to count as a rally.

Tests (`tests/`) ? one file per eval module

- `@tests/test_eval_metrics.py` ? metrics.py (IoU, matching, scoring, pr_curve)
- `@tests/test_eval_annotations.py` ? annotations.py (timestamp parsing, CSV load/validation)
- `@tests/test_calibration.py` ? calibration.py (kfold, select_best, cross_validate, leave_one_video_out, improvement_report)
- `@tests/test_calibrate_wasb.py` ? calibrate_wasb.py (load_golden_gts, predict_fn, grid, run)
- `@tests/test_calibrate_local.py` ? calibrate_local.py (local grid + gate, leave-one-video-out)

- `@tests/test_export_wasb_segments.py` ? export_wasb_segments.py (segment/comparison CSV schemas, mmss)
- `@tests/test_eval_harness.py` ? harness.py (stage prediction fetch, evaluate, report formatting)
- `@tests/test_eval_batch.py` ? batch.py (path resolution, stage selection, aggregate)
- `@tests/test_eval_regression.py` ? regression.py (baseline store, regress, provider wiring)
- `@tests/test_eval_training.py` ? training.py (label_candidates, feature extraction, build_training_set)
- `@tests/test_eval_classifier.py` ? classifier.py (fit/predict_proba, balanced weights, JSON round-trip)
- `@tests/test_distill_local.py` ? distill_local.py (candidate building, predict_rallies, learned-vs-baseline)
- `@tests/test_gemini_refine.py` ? gemini_refine.py (merge_overlaps, offset mapping, refine flow)
- Adjacent (consumed by eval but owned elsewhere): `@tests/test_hit_detector.py` (audio hit_detector used by e1_probe), `@tests/test_rally_gate.py` (rally_gate used by calibrate_local/distill_local/export), `@tests/test_annotation_providers.py` + `@tests/test_automated_provider.py` (annotation_registry used by regression.py). No dedicated test file exists for `audio/e1_probe.py` (diagnostic-only script).

Cross-module dependency notes (footguns when refactoring)

- ? `metrics.Interval`/`match_intervals`/`score` are the spine ? imported by harness, calibration, training, distill_local, export, e1_probe.
- ? `calibrate_wasb.{load_golden_gts, make_predict_fn, BASELINE}` is a de-facto shared API for calibrate_local, export_wasb_segments, gemini_refine, e1_probe ? changing its golden-CSV schema (`start_time`/`end_time`) or `BASELINE` ripples widely.
- ? two distinct GT CSV schemas coexist: `annotations.py` (`start,end`) vs `calibrate_wasb.load_golden_gts` (`start_time,end_time`, also `export_wasb_segments` write schema) ? don't cross-feed.
- ? `gemini_refine.merge_overlaps` is reused by `distill_local.predict_rallies` (gap_tol=1.0); `harness.get_predictions` is reused by `regression.regress`.